Republic of Tunisia
Ministry of Higher Education and Scientific Research
Esprit School of Engineering

FoodRescue TN

Food-waste reduction: from surplus to someone's table

Technical Design Report

Course: Distributed Web Application

Team members and microservices

Student	Microservice
Malek Hammami	Donation Offer Service
Med Sherif	Matching & Pickup Service
Salma Boubakri	Partner Registry & Trust Service
Yasmine Ben Abdelkader	Impact Analytics Service
Rayen Ghileb	Volunteer Driver Service
Mohamed Fadhel	Surprise Basket Marketplace Service
Shared work: User & Auth + Notification, API Gateway, Eureka, Config Server, RabbitMQ (whole team)	

Supervisor: [Badiaa Boudid]

Academic year: 2026

Abstract

Every day, large quantities of edible food from bakeries, restaurants, hotels and supermarkets are thrown away, while families and charities in Tunisia lack food and landfilled waste emits methane. **FoodRescue TN** is a distributed web platform that routes this surplus to those who need it: donors publish surplus offers, the system matches each offer with the nearest compatible association, schedules the pickup with a volunteer driver, and measures the resulting impact in kilograms of food saved, meals provided and CO_2e avoided. Citizens can also buy discounted "surprise baskets" made from unsold stock. The solution follows a microservice architecture of six business services (Donation Offer, Matching & Pickup, Partner Registry & Trust, Impact Analytics, Volunteer Driver, Surprise Basket Marketplace), each with its own database (MongoDB, PostgreSQL/PostGIS, Redis) and built with Spring Boot, supported by an API Gateway, Eureka, a Config Server, Keycloak and RabbitMQ. Services communicate synchronously through REST/OpenFeign for validations and commands, and asynchronously through RabbitMQ events for facts and notifications. The project is managed with the Scrum agile methodology, organised in short sprints with a contract-first approach to APIs and events. This report presents the topic, the project methodology, the actors and features, the breakdown into microservices, the data of each service, their interactions and the overall architecture.

Keywords: food waste, microservices, Spring Boot, RabbitMQ, Keycloak, geo-matching, event-driven architecture, Scrum.

Contents

Abstract **i**

List of Figures **v**

List of Tables **vi**

1 Introduction and Project Description **1**
1.1 Context . . . 1
1.2 Problem Statement . . . 1
1.3 Proposed Solution . . . 2
1.4 Objectives . . . 2

2 Project Management Methodology **3**
2.1 Introduction . . . 3
2.2 Agile Methodology . . . 3
2.3 The Scrum Framework . . . 3
2.3.1 General presentation . . . 3
2.3.2 Why Scrum for FoodRescue TN . . . 3
2.3.3 Scrum roles . . . 4
2.3.4 Scrum artifacts . . . 4
2.3.5 Scrum events . . . 4
2.4 Conclusion . . . 4

3 Requirements Specification and Project Planning **6**
3.1 Introduction . . . 6
3.2 Requirement Analysis . . . 6
3.2.1 Actors Identification . . . 6
3.2.2 Global Use Case Diagram . . . 7
3.2.3 Functional Requirements . . . 7
3.2.4 Non-Functional Requirements . . . 9
3.3 Project Planning . . . 9
3.3.1 Product Backlog . . . 9
3.3.2 Identification of Scrum Roles . . . 11
3.3.3 Sprint Planning . . . 11
3.4 Work Environment . . . 11
3.4.1 Software Development Environment . . . 11
3.5 Conclusion . . . 16

4 Microservice Architecture **17**
4.1 Why a Microservice Architecture . . . 17
4.2 Decomposition Principle . . . 18
4.3 Service Catalogue . . . 19
4.3.1 Donation Offer Service . . . 19
4.3.2 Matching & Pickup Service . . . 20
4.3.3 Partner Registry & Trust Service . . . 20
4.3.4 Impact Analytics Service . . . 21
4.3.5 Volunteer Driver Service . . . 21
4.3.6 Surprise Basket Marketplace Service . . . 21
4.3.7 Choice between donation and surprise basket . . . 22
4.4 Infrastructure Services . . . 22
4.5 Technology Choices . . . 22
4.6 Main Risks and Mitigation . . . 23
4.7 Chapter Conclusion . . . 23

5 Data Managed by Each Service **26**
5.1 Donation Offer Service (MongoDB) . . . 26
5.2 Matching & Pickup Service (PostgreSQL + PostGIS) . . . 28
5.3 Partner Registry & Trust Service (PostgreSQL) . . . 29
5.4 Impact Analytics Service (PostgreSQL) . . . 31
5.5 Volunteer Driver Service (PostgreSQL) . . . 31
5.6 Surprise Basket Marketplace Service (PostgreSQL) . . . 32
5.7 User & Auth and Notification (Keycloak / PostgreSQL) . . . 33
5.7.1 Identity data in Keycloak . . . 34
5.7.2 Application data in PostgreSQL . . . 34
5.8 Data Ownership Summary . . . 34

6 Interactions between Services **36**
6.1 Communication Principles . . . 36
6.2 Synchronous Interactions (REST / OpenFeign) . . . 37
6.2.1 Main synchronous interactions . . . 38
6.2.2 REST API contracts . . . 38
6.2.3 Example of OpenFeign communication . . . 39
6.3 Asynchronous Interactions (RabbitMQ Events) . . . 40
6.3.1 RabbitMQ communication model . . . 40
6.3.2 Domain events in FoodRescue TN . . . 40
6.3.3 Advantages of asynchronous communication . . . 41
6.4 Sequence Diagrams . . . 42
6.4.1 Main flow: from offer to impact . . . 42
6.4.2 Surprise basket flow . . . 44
6.5 Reliability Aspects . . . 44
6.5.1 Timeouts and error handling . . . 46

6.5.2 Message delivery and retries . . . 46
6.5.3 Idempotent event consumption . . . 47
6.5.4 Data consistency and concurrent operations . . . 47
6.5.5 Reliability measures by service . . . 48
6.6 Chapter Conclusion . . . 48

7 Overall Architecture **50**
7.1 Global Architecture Diagram . . . 50
7.2 Explanation of the Request Path . . . 52
7.3 Deployment View . . . 55

8 Other Design Elements **59**
8.1 Security (Keycloak, JWT, Roles) . . . 59
8.1.1 Authentication with Keycloak . . . 59
8.1.2 JSON Web Token (JWT) . . . 60
8.1.3 Role-Based Access Control (RBAC) . . . 60
8.1.4 Authorization and Access Control . . . 61
8.1.5 Security Objectives . . . 61
8.2 Business Rules and Assumptions . . . 61
8.2.1 User and Partner Management . . . 62
8.2.2 Donation Offer Management . . . 62
8.2.3 Matching and Pickup Management . . . 62
8.2.4 Surprise Basket Marketplace . . . 63
8.2.5 Ratings and Impact Calculation . . . 63
8.2.6 Main Business Assumptions . . . 64
8.3 Team Organization . . . 64
8.3.1 Development Responsibilities . . . 64
8.3.2 Work Organization . . . 65
8.3.3 Collaboration and Coordination . . . 65
8.4 Technology Stack Summary . . . 66
8.4.1 Database Allocation . . . 66
8.4.2 Technology Selection . . . 66

General Conclusion **70**
Conclusion . . . 71

A Event Contract Examples **72**

List of Figures

2.1 The Scrum cycle applied to FoodRescue TN 5

3.1 Global use case diagram of FoodRescue TN 7
3.2 Gantt chart of the project (W1 to W7: weeks from the project start) . . . 14

4.1 Decomposition of FoodRescue TN into bounded contexts 19

6.1 General asynchronous communication through RabbitMQ 40
6.2 Main sequence: from offer publication to impact calculation 43
6.3 Surprise basket sequence: from order to impact calculation 45

7.1 Overall architecture of FoodRescue TN 51

List of Tables

2.1 The three Scrum roles . . . 4
2.2 Scrum events . . . 5

3.1 Actors of FoodRescue TN . . . 6
3.2 Functional requirements . . . 7
3.3 Non-functional requirements . . . 9
3.4 Product backlog . . . 9
3.5 Scrum roles in the FoodRescue TN team . . . 11
3.6 Sprint plan . . . 12
3.7 Work of each service owner per sprint . . . 13
3.8 Software development environment . . . 15

4.1 Catalogue of the business microservices . . . 19
4.2 Infrastructure and shared services . . . 23
4.3 Technology choices and justification . . . 24
4.4 Main risks and mitigation measures . . . 24

5.1 Collection `offers` of the Donation Offer service . . . 27
5.2 Tables of the Matching & Pickup service . . . 28
5.3 Tables of the Partner Registry & Trust service . . . 30
5.4 Tables of the Impact Analytics service . . . 31
5.5 Tables of the Volunteer Driver service . . . 32
5.6 Tables of the Surprise Basket Marketplace service . . . 33
5.7 Tables of the User & Auth and Notification service . . . 34
5.8 Data ownership by service . . . 35
5.9 References between services (identifiers without foreign keys) . . . 35

6.1 Comparison of communication mechanisms . . . 37
6.2 Main synchronous interactions between services . . . 38
6.3 REST endpoints used in inter-service communication . . . 39
6.4 Main RabbitMQ events in FoodRescue TN . . . 41
6.5 Reliability measures applied to FoodRescue TN . . . 48

7.1 Main components of the FoodRescue TN deployment . . . 56

8.1 Main user roles and permissions . . . 68
8.2 Technology stack of FoodRescue TN . . . 69

Chapter 1

Introduction and Project Description

1.1 Context

Food is lost or wasted at every stage of the supply chain, from production to the final consumer. In Tunisia, as in many countries of the Mediterranean region, bakeries, restaurants, hotels, supermarkets and event caterers throw away every day edible food that is still safe to eat: unsold bread, prepared meals that were not served, fresh produce and packaged products close to their expiry date. Most of this surplus ends up in landfills, where the decomposition of organic matter emits methane, a greenhouse gas far more potent than carbon dioxide over the short term. Wasting food also wastes the water, energy and labour used to produce it.

At the same time, many families face food insecurity, and a large network of associations and charities works to feed them with limited resources, limited transport and little visibility on where surplus is available. Donors are generally willing to give, but they lack a simple and fast channel to do so before the food spoils.

1.2 Problem Statement

The surplus of donors and the needs of associations exist in the same cities, but nothing connects them in time. The problem can be summarised in four points.

- **No channel for donors:** a bakery or a hotel that wants to give its surplus has no simple way to find a charity, and the pickup window is short (often a few hours).
- **No coordination for associations:** associations cannot know which surplus is available nearby, nor organise the transport with limited volunteers.
- **No trust or traceability:** the quality of the food, the reliability of the partners and the path of each donation are not recorded.
- **No measure of the impact:** nobody can tell how many kilograms of food, how many meals and how much CO_2e were actually saved.

The central question of the project is therefore: *how can a surplus of edible food be routed, quickly and in a traceable way, from the donor who has it to the association or citizen who needs it, and how can the result be measured?*

1.3 Proposed Solution

FoodRescue TN is a distributed web platform that follows the surplus from the donor to someone's table. A donor publishes a surplus offer (type of food, quantity, pickup window, photo). The platform matches the offer with the nearest compatible association, schedules the pickup with an available volunteer driver, and records the delivery. Food that is not suited to donation can instead be sold at a reduced price to citizens as "surprise baskets". Every delivery and every collected basket feeds an impact dashboard showing the kilograms of food saved, the meals provided and the CO_2e avoided. Partners are verified, and the quality of each exchange is tracked through food-safety checklists and ratings.

The platform is built as a microservice architecture of six business services, each owned by one team member and holding its own database, supported by shared infrastructure services (API Gateway, Eureka, Config Server, Keycloak, RabbitMQ) and a front-end application consuming the APIs exposed through the gateway.

1.4 Objectives

The project pursues the following objectives.

- **Reduce food waste** by giving donors a fast and simple way to publish their surplus.
- **Route food quickly** to the most suitable association through geographic matching and volunteer drivers.
- **Build trust and traceability** through partner verification, food-safety checklists, ratings and status history.
- **Measure the impact** in kilograms saved, meals provided and CO_2e avoided, per governorate and per donor.
- **Apply the course concepts** of distributed web applications: microservices, API Gateway, service discovery, centralised configuration, secured access, synchronous and asynchronous communication, and containerisation.

Chapter 2

Project Management Methodology

2.1 Introduction

The way a project is organised has a direct effect on its success, especially for a distributed application developed in parallel by six people under a fixed deadline. This chapter presents the methodology chosen by the team: the Agile approach, and within it the Scrum framework.

2.2 Agile Methodology

Agile is a family of iterative and incremental approaches to software development. Instead of specifying everything in advance and delivering at the end, as in a sequential (waterfall) model, an agile team delivers the product in short cycles, collects feedback after each cycle and adapts its plan. The *Agile Manifesto* summarises this philosophy through four values:

- individuals and interactions over processes and tools;
- working software over comprehensive documentation;
- customer collaboration over contract negotiation;
- responding to change over following a plan.

2.3 The Scrum Framework

2.3.1 General presentation

Scrum is the most widely used agile framework. It organises the work in fixed-length iterations called *sprints*, at the end of which the team delivers a potentially usable product increment. A Scrum team is small, self-organised and cross-functional, and it relies on three pillars: transparency of the work, regular inspection of the product and of the process, and adaptation when something deviates.

2.3.2 Why Scrum for FoodRescue TN

Scrum fits this project for the following reasons.

- **Changing requirements:** matching rules, event payloads and the choice between donation and surprise basket will evolve during the project.
- **Incremental services:** each microservice starts minimal (CRUD) and grows step by step (communication, events, security).
- **Early integration:** each sprint review forces the independently developed services to work together.
- **Visible progress:** the sprint board and the reviews make the progress of every service visible to the whole team and to the Product Owner.

2.3.3 Scrum roles

Scrum defines three roles, summarised in Table 2.1. Their assignment to our team is given in Chapter 3.

Table 2.1: The three Scrum roles

Role	Responsibility
Product Owner	Represents the stakeholders, owns the product backlog, defines priorities and acceptance criteria, and validates the increments.
Scrum Master	Ensures that Scrum is understood and applied, facilitates the events and removes the obstacles blocking the team.
Development Team	Designs, implements, tests and documents the product; it is self-organised and jointly responsible for the increment.

2.3.4 Scrum artifacts

- **Product backlog:** the ordered list of everything the product needs, written as user stories and prioritised by the Product Owner.
- **Sprint backlog:** the items selected for the current sprint, with the tasks needed to deliver them.
- **Increment:** the working result of a sprint, which meets the *Definition of Done*.

2.3.5 Scrum events

Table 2.2 lists the events of a sprint and Figure 2.1 shows the resulting cycle.

2.4 Conclusion

Scrum gives the team a light but disciplined way to work: short sprints, a prioritised backlog and regular reviews. The next chapter applies it to FoodRescue TN by specifying the requirements, building the product backlog and planning the sprints.

Table 2.2: Scrum events

Event	**Frequency**	**Purpose**
Sprint	One to two weeks	Container of all the other events; produces an increment.
Sprint planning	Start of each sprint	Select the backlog items and define the sprint goal.
Daily stand-up	Short (15 minutes), two to three times a week in our case	Share what was done, what is next and what blocks.
Sprint review	End of each sprint	Demonstrate the increment and collect feedback.
Sprint retrospective	After the review	Identify what went well and what to improve.

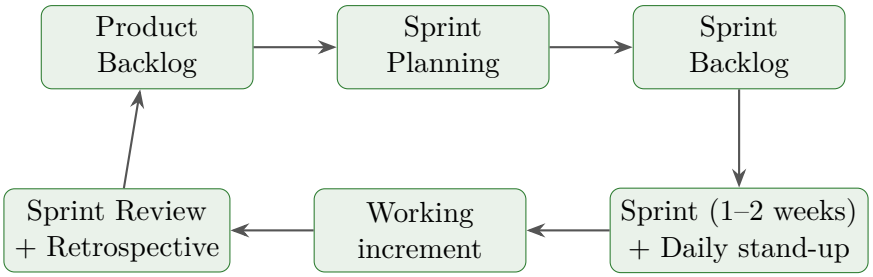


Figure 2.1: The Scrum cycle applied to FoodRescue TN

Chapter 3

Requirements Specification and Project Planning

3.1 Introduction

This chapter specifies what FoodRescue TN must do and how the work is organised. It first analyses the requirements (actors, global use case diagram, functional and non-functional requirements). It then presents the project planning following Scrum (product backlog, Scrum roles, sprint planning and Gantt chart). It ends with the work environment used by the team.

3.2 Requirement Analysis

3.2.1 Actors Identification

An actor is a person or a system that interacts with the platform. Table 3.1 lists the five actors of FoodRescue TN.

Table 3.1: Actors of FoodRescue TN

Actor	Description	Main goals
Donor	Bakery, restaurant, hotel or shop that has edible surplus.	Publish surplus quickly, avoid waste, sell unsold stock as baskets, see the impact of its donations.
Association / charity	Verified organisation that distributes food to people in need.	Receive compatible surplus nearby, confirm and follow pick-ups, check food safety, rate donors.
Volunteer driver	Volunteer who transports food from donors to associations.	Declare availability, be assigned to nearby pickups, update delivery status.
Citizen	Person who buys discounted surprise baskets.	Save money on good food, reduce waste, order a basket and collect it with a pickup code.
Admin	Administrator of the platform.	Verify partners, manage users, supervise the platform and monitor the impact.

3.2.2 Global Use Case Diagram

Figure 3.1 shows the main use cases of each actor. Registration and login through Keycloak are available to every actor and are not drawn, to keep the diagram readable.

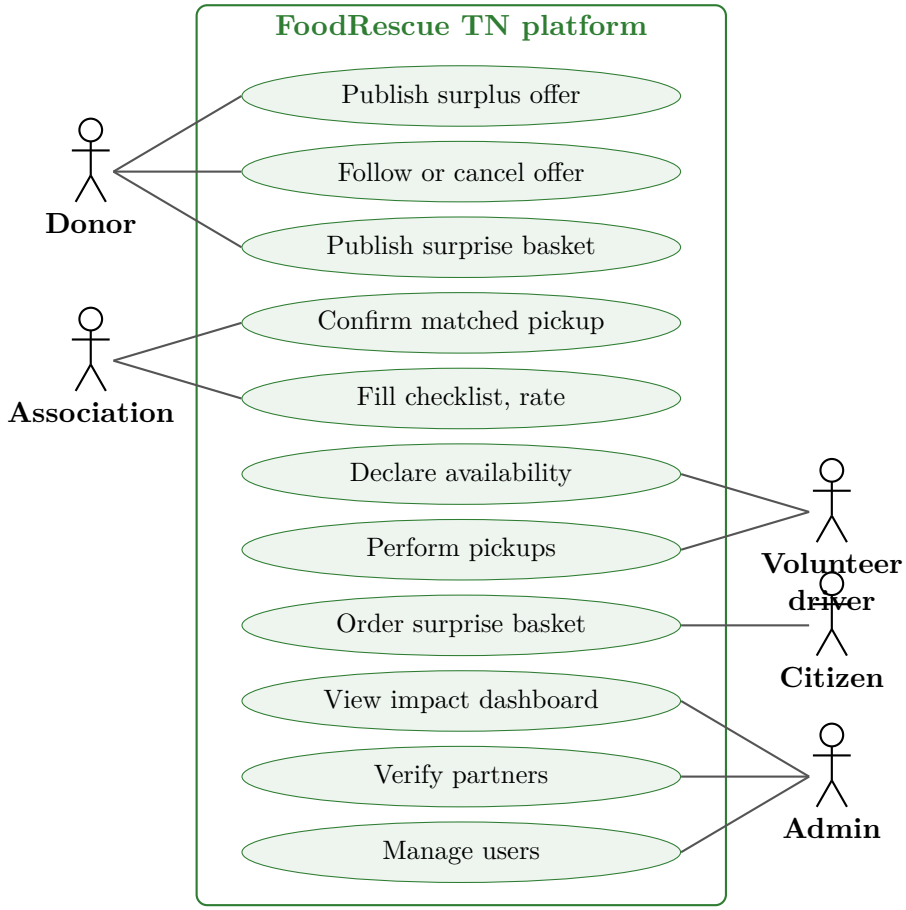


Figure 3.1: Global use case diagram of FoodRescue TN

3.2.3 Functional Requirements

Table 3.2 lists the functional requirements, the actor concerned and the service responsible. They are the source of the product backlog of Section **??**.

Table 3.2: Functional requirements

ID	Requirement	Actor	Service
FR1	Register and log in with a role (donor, association, driver, citizen, admin)	All	User & Auth (Keycloak)
FR2	Create and edit a partner profile (address, accepted food categories, capacity)	Donor, Association	Partner Registry & Trust
FR3	Verify or reject a partner	Admin	Partner Registry & Trust

ID	Requirement	Actor	Service
FR4	Publish a surplus offer (type, quantity, pickup window, photo, location)	Donor	Donation Offer
FR5	Edit or cancel an offer; automatic expiry at the end of the pickup window	Donor	Donation Offer
FR6	Browse offers by category and status	Association, Admin	Donation Offer
FR7	Match an offer with the nearest compatible association	Association (automatic)	Matching & Pickup
FR8	Confirm or reject a proposed match	Association	Matching & Pickup
FR9	Schedule a pickup and track its status until delivery	Association, Driver	Matching & Pickup
FR10	Declare availability slots, vehicle and zone	Driver	Volunteer Driver
FR11	Find and assign an available driver near the pickup point	Driver (automatic)	Volunteer Driver, Matching & Pickup
FR12	Update the status of a pickup (in transit, delivered)	Driver	Matching & Pickup
FR13	Fill in a food-safety checklist when receiving a delivery	Association	Partner Registry & Trust
FR14	Rate and review a partner or a driver after an exchange	Association, Donor	Partner Registry & Trust, Volunteer Driver
FR15	Publish a surprise basket built from unsold stock	Donor	Surprise Basket Marketplace
FR16	Browse baskets, order one (simulated payment) and get a unique pickup code	Citizen	Surprise Basket Marketplace
FR17	Collect a basket with its pickup code	Citizen, Donor	Surprise Basket Marketplace
FR18	Compute kg saved, meals provided and CO_2e avoided from deliveries and collected baskets	System	Impact Analytics
FR19	Display the impact dashboard per governorate and per donor, and the leaderboard	Admin, Donor	Impact Analytics
FR20	Produce a monthly impact report	Admin	Impact Analytics
FR21	Send notifications (e-mail or push) on key events, for example a confirmed match	All	Notification
FR22	(Optional) Forecast the demand for surplus	Admin	Impact Analytics (FastAPI)

3.2.4 Non-Functional Requirements

Table 3.3 presents the quality requirements of the platform.

Table 3.3: Non-functional requirements

Requirement	Description
Security	Authentication with Keycloak (OAuth2 / OpenID Connect), JWT tokens checked at the API Gateway, role-based access to every endpoint, no password handled by the business services.
Scalability	Each service can be scaled independently; clients find instances through Eureka.
Availability and fault tolerance	The failure of one service (for example Impact Analytics) must not block the main flow; resilience mechanisms (timeouts, retries, circuit breaker with Resilience4j) protect synchronous calls.
Loose coupling	Services do not share databases and exchange facts through RabbitMQ events.
Performance	Matching and dashboard queries must stay responsive; geospatial indexes and pre-computed aggregates are used for this purpose.
Traceability	Status changes of pickups and orders are recorded with their date and author.
Usability	A responsive front-end, with a simple publication flow for donors.

3.3 Project Planning

3.3.1 Product Backlog

Table 3.4: Product backlog

ID	Service	User story	Priority	Sprint
US-01	Donation Offer	As a donor, I want to publish a surplus offer (type, quantity, pickup window, photo) so that an association can collect it.	Must	1
US-02	Donation Offer	As a donor, I want to edit or cancel an offer that is not yet locked so that I can correct my mistakes.	Should	1
US-05	Partner Registry	As a donor or an association, I want to create my profile and have it verified by an admin so that partners can trust me.	Must	1

ID	Service	User story	Priority	Sprint
US-07	Volunteer Driver	As a volunteer driver, I want to declare my availability slots so that I can be assigned to pickups.	Must	1
US-09	Surprise Basket	As a donor, I want to publish a surprise basket from my unsold stock so that I can sell it at a reduced price.	Should	1
US-03	Matching & Pickup	As an association, I want to be matched with the nearest compatible offer so that food is collected quickly.	Must	2
US-08	Matching & Pickup, Volunteer Driver	As a volunteer driver, I want to be assigned to a nearby pickup and update its status so that deliveries are tracked.	Must	2
US-10	Surprise Basket	As a citizen, I want to order a discounted basket so that I get a unique pickup code.	Should	2
US-15	User & Auth	As a user, I want to register and log in with my role so that I only access what concerns me.	Must	2
US-04	Matching & Pickup	As an association, I want to confirm a match and follow the pickup status so that I know when the food arrives.	Must	3
US-16	Notification	As an association or a driver, I want to be notified when a match is confirmed so that I can act immediately.	Should	3
US-06	Partner Registry	As an association, I want to fill in a food-safety checklist and rate the donor so that trust is built.	Should	4
US-11	Surprise Basket	As a citizen, I want to collect my basket with the pickup code so that the sale is completed.	Should	4
US-12	Impact Analytics	As an admin, I want a dashboard of kg saved, meals and CO_2e avoided so that the impact is measurable.	Should	4
US-13	Impact Analytics	As an admin, I want a monthly report and a leaderboard so that I can communicate the results.	Could	4

ID	Service	User story	Priority	Sprint
US-14	Impact Analytics	As an admin, I want a demand forecast so that associations can anticipate surplus.	Could	5

3.3.2 Identification of Scrum Roles

Table 3.5 assigns the Scrum roles in our team.

Table 3.5: Scrum roles in the FoodRescue TN team

Role	Assigned to	Remark
Product Owner	charity donor	Validates each increment and sets the priorities with the team.
Scrum Master	Mrs Badiaa bouhdid	Facilitates the events and removes obstacles.
Development Team	Malek Hammami, Med Sherif, Salma Boubakri, Yasmine Ben Abdelkader, Rayen Ghileb, Mohamed Fadhel	Each member owns one business microservice; the infrastructure and the common User & Auth + Notification service are shared by all.

3.3.3 Sprint Planning

The project is planned as a seven-week delivery split into six sprints: a two-week inception sprint (Sprint 0) followed by five one-week development sprints (Sprints 1 to 5). Sprint 0 is dedicated to the design: no business code is written before the design is reviewed by the Product Owner. The short development sprints suit the limited delivery time, since they give an integrated increment to review every week. Each sprint starts with a planning meeting and ends with a review and a retrospective. Table 3.6 gives the goal and the main work of each sprint, and Table 3.7 details the work of each service owner.

Gantt chart. Figure 3.2 shows the schedule of the sprints and of the main tasks over the seven weeks of the project. The front-end and the documentation run in parallel with the development of the services.

3.4 Work Environment

3.4.1 Software Development Environment

Table 3.8 lists the technologies and tools used, grouped by purpose. They follow the mandatory stack of the module.

Table 3.6: Sprint plan

Sprint	**Weeks**	**Goal**	**Main work and increment**
Sprint 0	W1 – W2	Inception and design	Topic, actors and features; breakdown into microservices; data model, REST and event contracts; architecture diagram; Git repository, project board and microservice projects. *Increment:* validated design and technical document.
Sprint 1	W3	Foundations and core services	Config Server, Eureka and API Gateway; Docker Compose for the infrastructure (databases, RabbitMQ, Keycloak); CI pipeline; CRUD, database and Swagger documentation of each service; front-end skeleton. *Increment:* all services start, register in Eureka and are reachable through the gateway.
Sprint 2	W4	Security and business flow	Keycloak realm, roles and JWT checked at the gateway; synchronous calls with OpenFeign and Resilience4j; nearest-association matching and driver assignment; basket orders with pickup codes; front-end screens for donors and associations. *Increment:* a donation can be published, matched and assigned to a driver.
Sprint 3	W5	Events and notifications	RabbitMQ exchanges, queues and events (`OfferPublished`, `MatchConfirmed`, `DeliveryCompleted`, `BasketOrdered`); idempotent consumers; Notification service; front-end screens for drivers and citizens. *Increment:* the donation flow runs end to end with events and notifications.
Sprint 4	W6	Trust and impact	Ratings and food-safety checklist; basket collection; events `RatingSubmitted` and `BasketCollected`; impact aggregation, dashboard, monthly report and leaderboard; front-end dashboard and basket screens. *Increment:* the complete functional scope.
Sprint 5	W7	Hardening and release	Failure scenarios (unavailable service, expired offer); integration and end-to-end tests; optional demand forecast; Dockerization of the whole stack; documentation and demo rehearsal. *Increment:* final release.

Table 3.7: Work of each service owner per sprint

Service (owner)	**Sprint 1**	**Sprint 2**	**Sprint 3**	**Sprint 4**	**Sprint 5**
Donation Offer (M. Hammami)	Offer CRUD, MongoDB indexes, Swagger	Donor check in Partner Registry; lock endpoint for Matching	Publish `OfferPublished`; consume `BasketOrdered`; expiry task	Cancel rules; offer status in the front-end	Integration tests, API documentation
Matching & Pickup (M. Sherif)	Match and pickup model, PostGIS setup	Nearest-association query; calls to Offer, Partner and Driver; driver assignment	Consume `OfferPublished`; publish `MatchConfirmed` and `DeliveryCompleted`; status history	Pickup status transitions and tracking endpoints	Failure scenarios (no driver, expired offer); tests
Partner Registry & Trust (S. Boubakri)	Partner profiles and verification workflow	Donor-check, capacity and location endpoints; role checks	Rating tables and trust-score model	Safety checklist, ratings, `RatingSubmitted` event	Integration tests, documentation
Impact Analytics (Y. Ben Abdelkader)	Data model, conversion factors, seed data	Impact REST endpoints on test data	Consume `DeliveryCompleted`; idempotent processing	Monthly aggregates, dashboard, report, leaderboard; consume `BasketCollected` and `RatingSubmitted`	Optional forecast (FastAPI); tests
Volunteer Driver (R. Ghileb)	Driver profile, availability slots, overlap constraint	Available-driver search; slot booking at assignment	Assignment history; slot release on cancellation	Driver ratings	Integration tests, documentation
Surprise Basket (M. Fadhel)	Basket catalogue CRUD	Orders, atomic stock decrease, unique pickup code, simulated payment	Publish `BasketOrdered`; expiry of baskets	Collection flow, `BasketCollected` event	Tests, documentation
Shared (whole team)	Config Server, Eureka, Gateway, Compose infrastructure, CI, front-end skeleton	Keycloak realm and JWT; front-end screens for donors and associations	RabbitMQ topology, event contracts, Notification service; front-end screens for drivers and citizens	Front-end dashboard and basket screens	Dockerization of the stack, end-to-end tests, documentation, demo

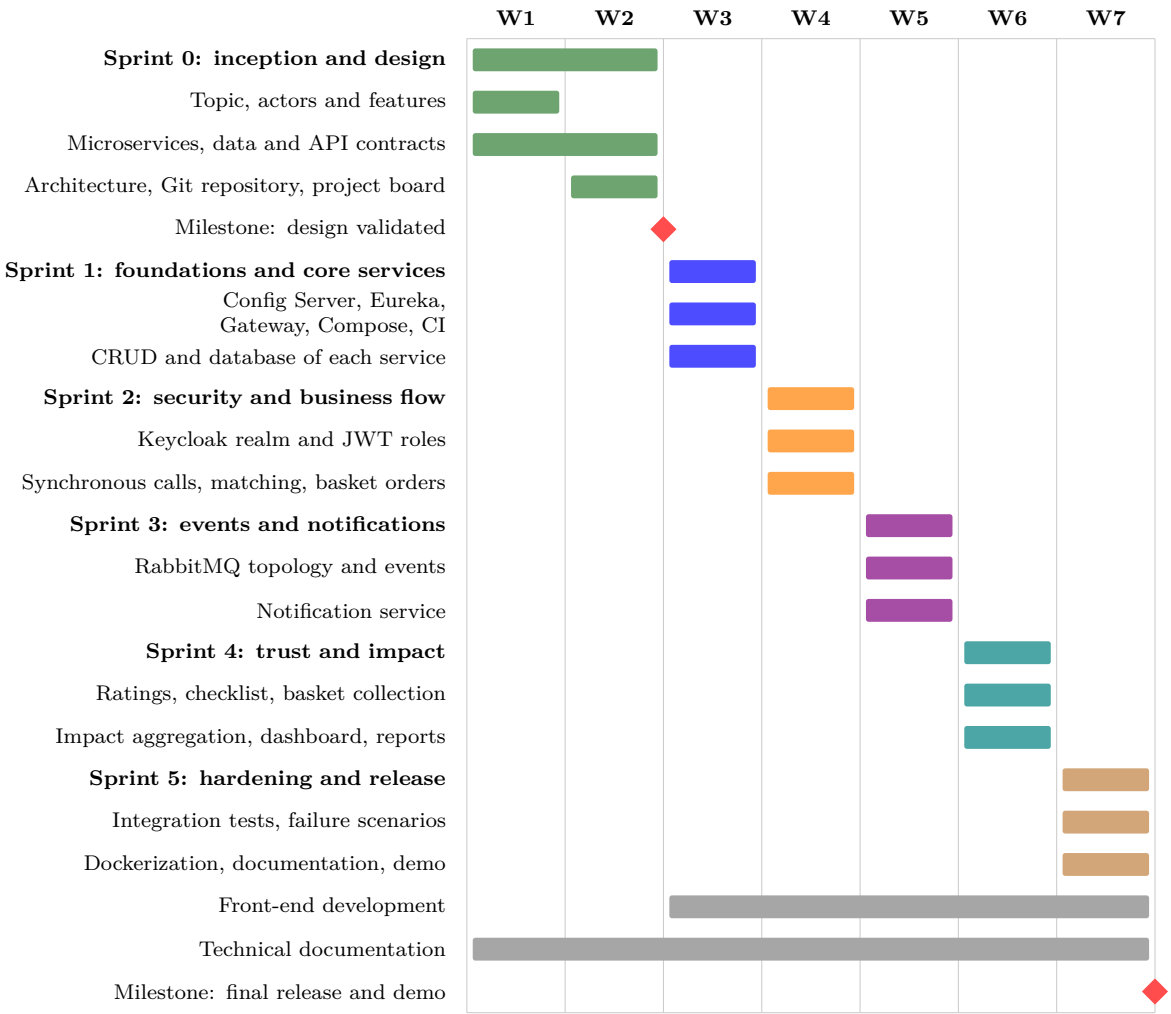


Figure 3.2: Gantt chart of the project (W1 to W7: weeks from the project start)

Table 3.8: Software development environment

Purpose	**Technology / tool**	**Usage in the project**
Back-end framework	Java 17, Spring Boot 3	Implementation of the six business microservices and the common service.
Spring Cloud	Spring Cloud Gateway, Eureka, Config Server, OpenFeign, Resilience4j	Single entry point, service discovery, centralised configuration, synchronous calls and resilience.
Messaging	RabbitMQ	Asynchronous domain events between services.
Security	Keycloak, OAuth2 / JWT	Authentication, roles and tokens.
Databases	PostgreSQL	One database per service; geospatial queries for the matching; flexible documents for offers.
Optional service	Python, FastAPI	Demand forecast of Impact Analytics.
Front-end	Angular, TypeScript, Leaflet, Chart.js	Web client consuming the APIs through the gateway: maps, charts and dashboards.
Containers and CI	Docker, Docker Compose, GitHub Actions	Reproducible environments, one-command startup, automated build and tests.
API documentation and tests	Swagger / OpenAPI, Postman, JUnit	Documentation of the endpoints, shared collection, unit and integration tests.
Version control	Git, GitHub	Source code of all services, branches and pull requests.

3.5 Conclusion

This chapter specified the requirements of FoodRescue TN: five actors, twenty-two functional requirements and the main quality requirements. It then organised the work with Scrum: a product backlog of user stories prioritised with MoSCoW, the assignment of the Scrum roles, six sprints (a two-week inception sprint and five one-week development sprints) with the work of each service owner, and a Gantt chart. Finally, it presented the hardware and software environment of the team. The next chapter explains how the platform is decomposed into microservices.

Chapter 4

Microservice Architecture and Service Decomposition

Domain-driven partitioning of FoodRescue TN into autonomous services, service catalogue and technology choices

This chapter explains how FoodRescue TN is decomposed into independent services. It first justifies the choice of a microservice architecture, then presents the principle used to draw the service boundaries, the catalogue of the six business services, the shared infrastructure services, the technology choices and finally the main risks of the architecture.

4.1 Why a Microservice Architecture

FoodRescue TN connects several actors (donors, associations, volunteer drivers, citizens and administrators) around business capabilities that differ strongly in nature: publishing offers, geographic matching, trust management, scheduling of drivers, statistical aggregation and e-commerce. These capabilities do not have the same data model, the same load profile or the same rate of change. A microservice architecture was therefore preferred to a monolithic application, for the following reasons.

- **Independent ownership and development:** each of the six team members owns one service end to end (code, database, tests, documentation). Services can be developed in parallel once the REST and event contracts are agreed.
- **Independent data model per service:** each service has its own database and schema, designed for its business needs: flexible documents for donation offers (MongoDB), geospatial queries for matching (PostgreSQL with PostGIS), and relational integrity and transactions for partners, drivers, impact aggregates and baskets (PostgreSQL). A change of schema in one service never affects the others.
- **Independent scalability:** the Donation Offer and Matching services receive far more traffic at peak hours than the Partner Registry. Each service can be scaled on its own.
- **Fault isolation:** a failure in one service (for example the Impact Analytics service) must not prevent donors from publishing offers or associations from collecting them.

- **Loose coupling:** services exchange facts through RabbitMQ events and never share a database, which limits the effect of a change in one service on the others.
- **Alignment with the course objectives:** the project is a distributed web application, and the decomposition exposes the real challenges of the domain: service discovery, centralised configuration, secured access, inter-service communication and eventual consistency.

The main drawbacks of this style are the additional complexity of distributed communication, the need for consistent contracts, and the difficulty of testing end-to-end flows. They are addressed by the contract-first approach and the integration tests presented in Chapter 2, by the reliability mechanisms described in Section 6.5, and by the mitigation measures of Section 4.6.

4.2 Decomposition Principle

The decomposition follows the **Domain-Driven Design** approach: the platform is divided into *bounded contexts*, each corresponding to a coherent business capability with its own vocabulary and its own data. The following rules were applied to define the boundaries.

1. **One business capability per service.** A service answers a single question of the domain (for example "what is being offered?" or "who should collect it?").
2. **Database per service.** Each service is the only owner of its data. No other service reads or writes its database directly: access goes through its REST API or through events.
3. **Single owner per service.** Each service is assigned to one team member, which clarifies responsibility and simplifies code review.
4. **Synchronous calls for questions and commands, events for facts.** A service calls another one through REST/OpenFeign when it needs an immediate answer (a validation, for instance). It publishes a RabbitMQ event when it announces something that has happened and other services may react to it.
5. **Cohesion over size.** A service is not split further as long as its data and rules change together. It is not merged with another service when its data lifecycle is different.

Applying these rules to the food rescue process gives the following chain of capabilities: *offer* the surplus, *trust* the partners, *match* offer and association, *deliver* with a volunteer, *measure* the impact, and *sell* the unsold stock as surprise baskets. Each element of this chain becomes one business service, as illustrated in Figure 4.1.

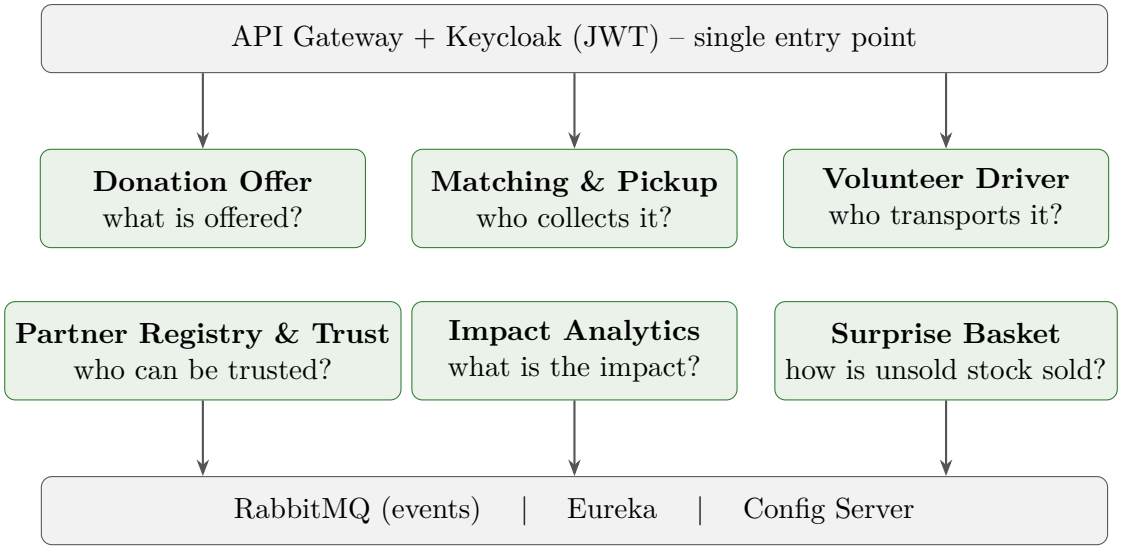


Figure 4.1: Decomposition of FoodRescue TN into bounded contexts

4.3 Service Catalogue

This section describes the six business services. Table 4.1 gives an overview, and each service is then detailed. The data models are presented in Chapter 5 and the interactions in Chapter 6.

Table 4.1: Catalogue of the business microservices

Service	Owner	Main responsibility	Database
Donation Offer	Malek Hammami	Publication and lifecycle of surplus offers	MongoDB
Matching & Pickup	Med Sherif	Geographic matching of offers and associations, pickup scheduling, delivery tracking	PostgreSQL + PostGIS
Partner Registry & Trust	Salma Boubakri	Profiles of donors and associations, food-safety checklist, ratings	PostgreSQL
Impact Analytics	Yasmine Ben Abdelkader	Computation and display of kg saved, meals provided and CO_2e avoided	PostgreSQL
Volunteer Driver	Rayen Ghileb	Driver profiles, availability slots, assignment history and ratings	PostgreSQL
Surprise Basket Marketplace	Mohamed Fadhel	Discounted baskets, orders, pickup codes	PostgreSQL

4.3.1 Donation Offer Service

This service allows donors (bakeries, restaurants, hotels, supermarkets) to publish surplus food offers and to manage them over time.

- **Responsibilities:** create, update and cancel offers (food type, quantity, pickup

window, photo, location); expire offers whose pickup window has passed; maintain the availability status and the remaining quantity; expose offers to authorised users.

- **Why MongoDB:** offers vary a lot from one donor to another (a tray of bread, a hotel buffet, a pallet of yoghurts). A document model stores these variable attributes and photos naturally, without a rigid schema, and supports fast filtering by category and status.
- **Key interactions:** checks with the Partner Registry service that the donor is verified; publishes the `OfferPublished` event, which triggers the matching process; consumes `BasketOrdered` to reduce the available quantity when part of the stock is sold as surprise baskets; is called by the Matching service to read and lock an offer.

4.3.2 Matching & Pickup Service

This service is the core of the platform. It connects each published offer with the most suitable association and organises the pickup.

- **Responsibilities:** find the nearest compatible association (distance, accepted food categories, capacity); confirm the match; schedule the pickup, assign a volunteer driver and track the delivery status until it is completed.
- **Why PostgreSQL with PostGIS:** the matching relies on geospatial queries (distance, radius search), provided by the PostGIS extension, and on transactional consistency of matches and pickups.
- **Key interactions:** consumes `OfferPublished`; reads and locks the offer in the Offer service; reads the capacity and location of associations in the Partner Registry; asks the Volunteer Driver service for an available driver near the pickup point; publishes `MatchConfirmed` (towards the Notification service) and `DeliveryCompleted` (towards Impact Analytics).

4.3.3 Partner Registry & Trust Service

This service maintains the identity and reliability of the partners taking part in the food chain.

- **Responsibilities:** manage donor and association profiles (category, address, accepted food types, capacity) and their verification; handle the food-safety checklist; manage ratings and reviews.
- **Why PostgreSQL:** the data is highly structured with strong relationships (partners, checklists, ratings) and requires integrity constraints.

- **Key interactions:** answers requests from the Donation Offer service (is the donor verified?) and from the Matching service (capacity and location of associations); publishes `RatingSubmitted`, consumed by Impact Analytics to update the partner score.

4.3.4 Impact Analytics Service

This service turns operational events into measurable indicators of social and environmental impact.

- **Responsibilities:** aggregate the quantities delivered; compute kilograms of food saved, number of meals provided and CO_2e avoided, per governorate and per donor; serve the global dashboard and the leaderboard; produce a monthly report. An optional demand-forecast component, written in FastAPI, can complete these indicators.
- **Why PostgreSQL:** it stores the facts received through events and the aggregates derived from them. Pre-computed aggregate tables, indexes and grouping queries (by governorate, donor and month) keep the dashboard and the leaderboard fast without an additional cache.
- **Key interactions:** is mostly event-driven; it consumes `DeliveryCompleted`, `BasketCollected` and `RatingSubmitted`, and does not need to be called by the other services, so its availability does not affect the main flow.

4.3.5 Volunteer Driver Service

This service manages the volunteers who transport food from donors to associations.

- **Responsibilities:** register driver profiles (vehicle type, zone); manage availability slots; keep the assignment history and the ratings of each driver.
- **Why PostgreSQL:** slots and assignments are relational data where overlapping and double booking must be prevented by constraints and transactions.
- **Key interactions:** is queried by the Matching & Pickup service to find an available driver near the pickup point; the assignment itself and the delivery status are handled by the Matching & Pickup service.

4.3.6 Surprise Basket Marketplace Service

This service offers a second outlet for unsold stock: donors can sell it at a reduced price to citizens in the form of “surprise baskets”.

- **Responsibilities:** publish a catalogue of baskets built from donor stock (content description, discounted price, pickup slot, quantity); handle citizen orders; generate a pickup code; simulate the payment.
- **Why PostgreSQL:** baskets and orders are linked records with a simple lifecycle. Transactions and constraints guarantee that the available quantity is decreased atomically (no overselling) and that each pickup code is unique. Variable details of a basket can be stored in a JSONB column.
- **Key interactions:** publishes `BasketOrdered`, consumed by the Offer service to reduce the available quantity, and `BasketCollected`, consumed by Impact Analytics to add the kilograms saved; the Notification service can use these events to inform the citizen.

4.3.7 Choice between donation and surprise basket

For a given surplus, the donor can choose the donation route (Donation Offer and Matching services) or the sale route (Surprise Basket service). Both routes end with a measurable contribution to the impact indicators, which is why Impact Analytics listens to the events of both: `DeliveryCompleted` for donations and `BasketCollected` for sales. The quantity of an offer is shared between the two routes through the `BasketOrdered` event, so that the same food is never counted twice.

4.4 Infrastructure Services

In addition to the six business services, the platform relies on shared infrastructure components, developed and configured by the whole team. They provide the cross-cutting capabilities that the business services should not implement themselves.

These components follow standard patterns of distributed systems: *API gateway*, *service discovery*, *externalised configuration*, *centralised authentication* and *publish/subscribe messaging*. Their interplay is shown in the global architecture in Chapter 7, and the security model is detailed in Section 8.1.

4.5 Technology Choices

The technologies were selected for their maturity, their integration with the Spring ecosystem taught in the course, and their fit with the needs of each service.

Five services use PostgreSQL, which reduces the learning effort since most members work with the same tools, query language and migration mechanism, and simplifies deployment: one PostgreSQL server in Docker Compose can host one separate database per service. The Donation Offer service uses MongoDB because its offers have a variable structure. Services remain independent because no database is shared: each service owns its data and exposes it only through its API and events. The price to pay is eventual

Table 4.2: Infrastructure and shared services

Component	**Role in the architecture**
API Gateway	Single entry point for all client requests. Routes requests to the right service, checks the JWT token, applies cross-cutting rules (CORS, rate limiting) and hides the internal topology from clients.
Eureka (Discovery)	Service registry. Each service registers at start-up, and the gateway and Feign clients find service instances by name instead of fixed addresses, which allows scaling and relocation.
Config Server	Centralised external configuration for all services, per environment (development, test, production), so that settings can be changed without rebuilding images.
Keycloak	Identity and access management. Authenticates users and issues JWT tokens carrying the roles `DONOR`, `ASSOCIATION`, `DRIVER`, `CITIZEN` and `ADMIN`.
RabbitMQ	Message broker for asynchronous communication. Delivers domain events (such as `OfferPublished`, `MatchConfirmed`, `DeliveryCompleted`) to the interested services without coupling the sender to the receivers.
User & Auth + Notification	Shared service for user accounts linked to Keycloak, and for notifications (e-mail or push) triggered by events, for example `MatchConfirmed` to inform the association and the driver.

consistency between services, handled through events and idempotent consumers (Section 6.5).

4.6 Main Risks and Mitigation

The decomposition into six services and shared infrastructure brings risks that were identified from the start of the project. Table 4.4 summarises them with the planned countermeasures.

4.7 Chapter Conclusion

This chapter presented the decomposition of FoodRescue TN into autonomous services. A microservice architecture was chosen because the platform combines capabilities with different data models, load profiles and rates of change, and because it allows six developers to work in parallel on well-defined boundaries.

The decomposition follows Domain-Driven Design: each bounded context became one business service, owned by one team member and holding its own database. The six resulting services cover the complete food rescue chain: *offer* (Donation Offer), *match* (Matching & Pickup), *trust* (Partner Registry & Trust), *deliver* (Volunteer Driver), *measure* (Impact Analytics) and *sell* (Surprise Basket Marketplace). They are supported by shared infrastructure components (API Gateway, Eureka, Config Server, Keycloak,

Table 4.3: Technology choices and justification

Concern	**Technology**	**Justification**
Service implementation	Spring Boot	Rapid development of REST services, large ecosystem, native integration with Spring Cloud.
Synchronous communication	REST with OpenFeign	Declarative clients, integration with Eureka for service lookup, simple contracts documented with OpenAPI.
Asynchronous communication	RabbitMQ	Reliable delivery, flexible routing with exchanges and queues, decoupling between producers and consumers.
Relational storage	PostgreSQL	Integrity constraints, transactions and a mature ecosystem (Spring Data JPA) for matches and pickups, partners, drivers, baskets and impact aggregates. One separate database per service.
Semi-structured attributes	PostgreSQL JSONB	Storage of attributes that vary between records (basket details, event payloads) without a second database technology.
Geospatial queries	PostGIS	Distance and radius computations for nearest-association matching.
Document storage	MongoDB	Flexible schema for donation offers, whose attributes vary from one donor to another (used only by the Donation Offer service).
Optional forecasting	FastAPI	Lightweight Python service for the optional demand forecast of Impact Analytics.
Security	Keycloak, JWT	Standard protocols (OAuth2 / OpenID Connect), role-based access, centralised user management.
Discovery and configuration	Eureka, Spring Cloud Config	Dynamic service location and centralised configuration.
Deployment	Docker, Docker Compose	Reproducible environments and one-command start of the whole system.

Table 4.4: Main risks and mitigation measures

Risk	**Impact**	**Mitigation**
Complexity of the matching logic	The Matching & Pickup service is the core of the platform and may delay the whole project.	Start with a simple nearest-first rule and improve it later (capacity, trust level, food categories).
Keycloak configuration	Blocks secured access for every service.	Prepare one shared realm with the five roles early and reuse it for all services.
Inter-service integration	Contract mismatches between services developed in parallel.	Shared Postman collection, contract-first REST and event definitions, mocked events.
Uneven workload between members	Some services are finished late and block the integration.	Weekly integration test and regular progress review.

RabbitMQ and the User & Auth + Notification service) that provide routing, discovery, configuration, security and messaging.

Five services rely on PostgreSQL (with the PostGIS extension for the geographic matching), while the Donation Offer service uses MongoDB for its flexible offer documents. Spring Boot, OpenFeign and RabbitMQ are used for synchronous and asynchronous communication. The price of this independence is eventual consistency between services, which is addressed by the reliability mechanisms described in Section 6.5. The main risks of the architecture (matching complexity, Keycloak setup, integration and uneven workload) are managed through the measures of Section 4.6.

The next chapter, Chapter 5, details the data managed by each service, including its entities, attributes and ownership.

Chapter 5

Data Managed by Each Service

Entities, attributes, constraints and ownership of the data held by each microservice of FoodRescue TN

This chapter describes the data managed by each service of the platform. For every service it presents the entities stored, their main attributes, the constraints that protect the data and the references to data owned by other services. The chapter ends with a summary of data ownership.

Four design rules apply to all services, in line with the decomposition principles of Chapter 4:

- **Database per service:** each service is the only one to read and write its own database. Other services obtain its data through its REST API or through events.
- **References by identifier:** when a service needs to designate data owned by another service (a donor, an offer, a driver), it stores only the identifier of that data. No foreign key crosses a service boundary, and no query joins tables of two services.
- **Snapshots for history:** data that must remain stable after the exchange is copied at the moment it is used (for example the coordinates of an association at the time of a match), so that later changes in the source service do not alter past records.
- **Idempotent event handling:** services that consume events keep the identifiers of the events already processed, so that a message delivered twice by RabbitMQ is applied only once.

Unless stated otherwise, identifiers of relational tables are UUIDs, dates are stored as `TIMESTAMPTZ`, and quantities are expressed in kilograms (kg).

5.1 Donation Offer Service (MongoDB)

The Donation Offer service stores one document per surplus offer in the collection `offers`. A document model was chosen because the description of a surplus varies strongly from one donor to another (bread, prepared meals, fresh produce), and the variable details can be kept in the same document as the offer without a rigid schema.

Table 5.1: Collection `offers` of the Donation Offer service

Field	Type	Description
`_id`	ObjectId	Unique identifier of the offer.
`donorId`	String (UUID)	Identifier of the donor in the Partner Registry service (reference only).
`title`	String	Short title of the offer.
`category`	String (enum)	`BAKERY`, `PREPARED_MEALS`, `FRESH_PRODUCE`, `DAIRY`, `PACKAGED`, `OTHER`.
`description`	String	Free description of the surplus.
`quantity`	Embedded document	Initial quantity: `value` and `unit` (kg, portions or units).
`estimatedWeightKg`	Number	Estimated total weight, used later for the impact indicators.
`availableQuantity`	Number	Quantity still available; decreased when baskets are ordered (`BasketOrdered` event).
`pickupWindow`	Embedded document	`start` and `end` of the period in which the food can be collected.
`location`	Embedded document	`address`, `governorate` and `point` (GeoJSON coordinates).
`photoUrl`	String	Address of the photo; the image file itself is not stored in the document.
`attributes`	Embedded document	Variable details: allergens, storage temperature, packaging.
`status`	String (enum)	`AVAILABLE`, `LOCKED`, `COLLECTED`, `EXPIRED`, `CANCELLED`.
`createdAt`, `updatedAt`	Date	Creation and last modification dates.

Lifecycle. An offer is created with the status `AVAILABLE`. It becomes `LOCKED` when the Matching service reserves it for an association, and `COLLECTED` once the pickup is completed. A scheduled task sets the status `EXPIRED` when the end of the pickup window has passed. A donor can cancel an offer that is not yet locked; cancelling a locked offer is refused with a conflict response (HTTP 409).

Indexes. A compound index on `(status, category)` supports the filtered listing of offers; a geospatial index (`2dsphere`) on `location.point` supports searches by area; an index on `pickupWindow.end` supports the expiry task; an index on `(donorId, createdAt)` supports the history of a donor.

5.2 Matching & Pickup Service (PostgreSQL + PostGIS)

The Matching & Pickup service stores the matches between offers and associations, the pickups derived from them and the history of their statuses. The PostGIS extension provides the geographic type `geography(Point,4326)` and the distance functions used for the nearest-association rule.

Table 5.2: Tables of the Matching & Pickup service

Table	Main columns	Description
`match`	`id`, `offer_id`, `association_id`, `donor_id`, `offer_point`, `association_point`, `distance_m`, `status`, `created_at`, `confirmed_at`	Link between one offer and one association. The two geographic points are snapshots taken at matching time. Status: `PROPOSED`, `CONFIRMED`, `REJECTED`, `CANCELLED`.
`pickup`	`id`, `match_id`, `driver_id`, `window_start`, `window_end`, `quantity_kg`, `status`, `delivered_at`	Collection of the food, created from a confirmed match. Status: `SCHEDULED`, `ASSIGNED`, `IN_TRANSIT`, `DELIVERED`, `FAILED`, `CANCELLED`.
`pickup_status_history`	`id`, `pickup_id`, `old_status`, `new_status`, `changed_by`, `changed_at`	Trace of every status change of a pickup.

Relations and constraints.

- A `pickup` belongs to exactly one `match` (foreign key with a uniqueness constraint), and each row of `pickup_status_history` belongs to one `pickup`.
- The columns `offer_id`, `association_id`, `donor_id` and `driver_id` are references to data of other services: they are not foreign keys.
- A partial unique index on `match(offer_id)` restricted to the active statuses (`PROPOSED`, `CONFIRMED`) prevents two active matches for the same offer; a violation returns HTTP 409.
- Status columns are protected by `CHECK` constraints, and transitions are validated by the service before they are written.

Geographic data. The columns `offer_point` and `association_point` carry a GiST index. The query for the nearest compatible association uses a radius filter followed by an ordering by distance, which is the simple nearest-first rule adopted for the first version.

5.3 Partner Registry & Trust Service (PostgreSQL)

The Partner Registry & Trust service stores the profiles of donors and associations, the result of their verification, the food-safety checklists and the ratings. These data are strongly structured and linked, which suits a relational database with integrity constraints.

Relations and constraints.

- `accepted_category`, `safety_checklist` and `rating` refer to `partner` through foreign keys.
- `owner_user_id` stores the identifier of the user account in Keycloak, which links a profile to the person who manages it.
- The email address of a partner is unique, and `score` is limited to the range 1 to 5 by a `CHECK` constraint.
- A unique constraint on `(rater_id, pickup_id)` allows one rating per partner and per exchange.
- `pickup_id` is a reference to the Matching service and is not a foreign key.

Trust score. The `trust_score` of a partner is computed from its ratings and recalculated when a rating is submitted. Each new rating is also published as a `RatingSubmitted` event for the Impact Analytics service.

Table 5.3: Tables of the Partner Registry & Trust service

Table	Main columns	Description
partner	id, owner_user_id, type, name, email, phone, address, governorate, latitude, longitude, capacity_kg, verification_status, verified_at, trust_score	Profile of a donor or an association. type is DONOR or ASSOCIATION; capacity_kg is used for associations only; verification_status is PENDING, VERIFIED or REJECTED.
accepted_category	partner_id, category	Food categories accepted by an association.
safety_checklist	id, partner_id, pickup_id, temperature_ok, packaging_ok, expiry_ok, hygiene_ok, comment, completed_at	Food-safety checklist filled in by an association when it receives a delivery.
rating	id, rater_id, rated_id, pickup_id, score, comment, created_at	Rating and review given by one partner to another after an exchange.

5.4 Impact Analytics Service (PostgreSQL)

The Impact Analytics service stores the facts received through events and the aggregates derived from them. It owns no operational data of the other services: every row comes from an event.

Table 5.4: Tables of the Impact Analytics service

Table	Main columns	Description
`impact_fact`	`id`, `event_id`, `source`, `donor_id`, `association_id`, `governorate`, `category`, `quantity_kg`, `meals`, `co2e_kg`, `occurred_at`	One row per delivery or collected basket. `source` is `DELIVERY` or `BASKET`.
`impact_monthly`	`year_month`, `governorate`, `donor_id`, `kg_saved`, `meals`, `co2e_kg`	Pre-computed totals per month, governorate and donor, used by the dashboard and the monthly report.
`partner_score`	`partner_id`, `average_score`, `ratings_count`, `updated_at`	Score of each partner, updated from `RatingSubmitted` events.
`conversion_factor`	`category`, `meals_per_kg`, `co2e_per_kg`	Parameters used to convert kilograms into meals and avoided CO_2e.
`processed_event`	`event_id`, `received_at`	Identifiers of the events already handled, which guarantees idempotent consumption.

Computation. When a `DeliveryCompleted` or `BasketCollected` event arrives, the service checks `processed_event`, inserts a row in `impact_fact` with the meals and CO_2e computed from `conversion_factor`, and updates the matching row of `impact_monthly` in the same transaction. The global figures, the figures per governorate, the figures per donor and the leaderboard are read from `impact_monthly`, whose indexes on `(governorate, year_month)` and `(donor_id, year_month)` keep these queries fast without an additional cache.

5.5 Volunteer Driver Service (PostgreSQL)

The Volunteer Driver service stores the profiles of the volunteers, their availability slots, the history of their assignments and their ratings. The risk of double booking makes constraints and transactions essential here.

Table 5.5: Tables of the Volunteer Driver service

Table	Main columns	Description
`driver`	`id`, `user_id`, `full_name`, `phone`, `vehicle_type`, `capacity_kg`, `governorate`, `latitude`, `longitude`, `active`, `average_rating`	Profile of a volunteer. `vehicle_type` is `CAR`, `VAN`, `MOTORBIKE` or `BICYCLE`; the position is the base of the driver.
`availability_slot`	`id`, `driver_id`, `start_time`, `end_time`, `status`	Period in which a driver can work. Status: `FREE` or `BOOKED`.
`assignment`	`id`, `driver_id`, `slot_id`, `pickup_id`, `assigned_at`, `status`, `completed_at`	History of the pickups given to a driver. Status: `ASSIGNED`, `COMPLETED` or `CANCELLED`.
`driver_rating`	`id`, `driver_id`, `pickup_id`, `score`, `comment`, `created_at`	Rating of a driver after a delivery.

Relations and constraints.

- `availability_slot`, `assignment` and `driver_rating` refer to `driver` through foreign keys, and `assignment` also refers to `availability_slot`.
- An exclusion constraint on `(driver_id, time range)` prevents two overlapping slots for the same driver.
- A unique constraint on `assignment(pickup_id)` for the active status prevents a pickup from receiving two drivers; a violation returns HTTP 409.
- `pickup_id` is a reference to the Matching service. The delivery status itself is held by the Matching service; this service keeps only the history of the assignments of each driver.

5.6 Surprise Basket Marketplace Service (PostgreSQL)

The Surprise Basket Marketplace service stores the catalogue of baskets and the orders of citizens. Transactions and constraints guarantee that the available quantity is decreased atomically (no overselling) and that every pickup code is unique.

Relations and constraints.

- `basket_order` refers to `basket` through a foreign key. The columns `donor_id`, `offer_id` and `citizen_id` are references to other services.

Table 5.6: Tables of the Surprise Basket Marketplace service

Table	Main columns	Description
basket	id, donor_id, offer_id, title, description, details, original_price, discounted_price, weight_kg, quantity_total, quantity_available, governorate, pickup_start, pickup_end, status	Surprise basket built from donor stock. details is a JSONB column for variable content. Status: PUBLISHED, SOLD_OUT, EXPIRED, CANCELLED.
basket_order	id, basket_id, citizen_id, quantity, total_price, pickup_code, payment_status, payment_reference, status, ordered_at, collected_at	Order of a citizen. The payment is simulated. Status: CONFIRMED, COLLECTED, CANCELLED, EXPIRED.

- A CHECK constraint keeps quantity_available between 0 and quantity_total; the decrease is made by a conditional update inside the order transaction, so two simultaneous orders cannot sell the last basket twice (a refused order returns HTTP 409).
- pickup_code has a unique constraint.
- The table is named basket_order because order is a reserved word in SQL.

Events. A confirmed order publishes BasketOrdered (the Donation Offer service reduces the available quantity), and the collection of the basket publishes BasketCollected with the weight weight_kg, which feeds the Impact Analytics service.

5.7 User & Auth and Notification (Keycloak / PostgreSQL)

This shared component combines two kinds of data: the identity data managed by Keycloak, and the application data of the notification service stored in PostgreSQL.

5.7.1 Identity data in Keycloak

Keycloak stores the user accounts, the credentials, the roles and the registered clients in its own database. Passwords never appear in the application services: they exchange only the JWT token issued by Keycloak, which carries the user identifier and one of the roles `DONOR`, `ASSOCIATION`, `DRIVER`, `CITIZEN` and `ADMIN`. The identifier of the Keycloak user is the common key used by the other services (`owner_user_id`, `user_id`, `citizen_id`).

5.7.2 Application data in PostgreSQL

Table 5.7: Tables of the User & Auth and Notification service

Table	Main columns	Description
`user_profile`	`id`, `keycloak_id`, `email`, `full_name`, `phone`, `role`, `notify_email`, `notify_push`, `created_at`	Application profile linked to a Keycloak account, with the notification preferences of the user.
`device_token`	`id`, `user_id`, `token`, `platform`, `registered_at`	Device registrations used for push notifications.
`notification`	`id`, `user_id`, `type`, `channel`, `title`, `body`, `payload`, `status`, `source_event_id`, `created_at`, `sent_at`	Notification triggered by an event, for example `MatchConfirmed`. `channel` is `EMAIL` or `PUSH`; status: `PENDING`, `SENT`, `FAILED` or `READ`; `payload` is `JSONB`.

Constraints. `keycloak_id` and `email` are unique in `user_profile`. A unique constraint on `(source_event_id, user_id, channel)` in `notification` prevents the same event from sending the same message twice to a user.

5.8 Data Ownership Summary

Table 5.8 summarises, for each service, the data it owns, the technology that stores it and the way other services can reach it.

Since no foreign key crosses a service boundary, the links between services are carried by identifiers. Table 5.9 lists these references and the service through which each one is resolved.

The consequence of this organisation is eventual consistency: a change made in one service becomes visible in the others only when the corresponding event has been processed. The mechanisms that handle this situation (idempotent consumers, retries) are described in Section 6.5, and the exchanges between the services are detailed in Chapter 6.

Table 5.8: Data ownership by service

Service	**Owner**	**Database**	**Data owned and access by other services**
Donation Offer	Malek Hammami	MongoDB	Offers and their status. Read through `/api/offers`; changes announced by `OfferPublished`.
Matching & Pickup	Med Sherif	PostgreSQL + PostGIS	Matches, pickups and status history. Read through `/api/pickups`; changes announced by `MatchConfirmed` and `DeliveryCompleted`.
Partner Registry & Trust	Salma Boubakri	PostgreSQL	Partner profiles, verification, checklists, ratings. Read through `/api/partners`; ratings announced by `RatingSubmitted`.
Impact Analytics	Yasmine Ben Abdelkader	PostgreSQL	Impact facts, monthly aggregates, partner scores. Read through `/api/impact`; fed only by events.
Volunteer Driver	Rayen Ghileb	PostgreSQL	Driver profiles, slots, assignment history, ratings. Read through `/api/drivers`.
Surprise Basket Marketplace	Mohamed Fadhel	PostgreSQL	Baskets, orders and pickup codes. Read through `/api/baskets` and `/api/orders`; announced by `BasketOrdered` and `BasketCollected`.
User & Auth + Notification	Whole team	Keycloak / PostgreSQL	Accounts and roles (Keycloak); profiles and notifications (PostgreSQL).

Table 5.9: References between services (identifiers without foreign keys)

Reference	**Held by**	**Resolved through**
`donorId`, `donor_id`	Offer, Matching, Basket, Impact	Partner Registry (`GET /api/partners/{id}`)
`association_id`	Matching, Impact	Partner Registry
`offer_id`	Matching, Basket	Donation Offer (`GET /api/offers/{id}`)
`driver_id`	Matching	Volunteer Driver
`pickup_id`	Partner, Driver	Matching & Pickup
`user_id`, `owner_user_id`, `citizen_id`	Partner, Driver, Basket, Notification	Keycloak (JWT claim)

Chapter 6

Interactions between Services

Synchronous and asynchronous communication mechanisms, event-driven workflows and reliability principles governing the interactions between FoodRescue TN microservices

This chapter explains how the microservices of FoodRescue TN communicate to execute the platform's main business processes. It first presents the communication principles adopted by the architecture, then describes synchronous interactions through REST and OpenFeign and asynchronous interactions through RabbitMQ events. Two sequence diagrams illustrate the main food donation flow and the surprise basket flow. Finally, the chapter presents the reliability mechanisms required to preserve consistency and continuity in a distributed environment.

6.1 Communication Principles

FoodRescue TN is composed of six independent business microservices, each responsible for its own data and business operations. Although this separation allows independent development and deployment, the services must communicate to complete processes that involve several business capabilities.

For example, publishing a food offer may trigger partner verification, geographic matching, driver selection, pickup scheduling, notification and impact calculation. These operations cannot be performed by a single service because each step belongs to a different bounded context.

The communication strategy follows two complementary mechanisms: **synchronous communication** and **asynchronous communication**.

- **Synchronous communication:** a service sends a request to another service and waits for its response. It is used when the requesting service needs an immediate result to continue its operation, such as checking whether a donor is verified or retrieving available drivers.
- **Asynchronous communication:** a service publishes an event announcing that a business action has occurred. Other services interested in that event consume it independently, without requiring the publishing service to wait for their processing.

The following rules govern communication between services:

1. **No direct database access:** a service never reads or modifies another service's database. Data is obtained through REST APIs or received through events.
2. **REST for immediate decisions:** synchronous requests are used for validations, data retrieval and operations requiring an immediate confirmation.
3. **Events for completed business actions:** RabbitMQ events announce facts such as the publication of an offer, the confirmation of a match or the completion of a delivery.
4. **Loose coupling:** event producers do not need to know the internal implementation or database structure of event consumers.
5. **Eventual consistency:** information may take a short time to become visible in another service because event processing is asynchronous.
6. **Secure communication:** external requests pass through the API Gateway and are protected using JWT tokens issued by Keycloak. Internal service requests must also respect the authentication and authorization rules of the architecture.

Table 6.1 summarises the two communication mechanisms and their use in the platform.

Table 6.1: Comparison of communication mechanisms

Criterion	**Synchronous**	**Asynchronous**
Technology	REST / OpenFeign	RabbitMQ
Communication	Request and response	Publish and consume events
Response	Immediate response expected	No immediate consumer response
Dependency	Requester depends on the response	Producer is decoupled from consumers
Main purpose	Validation and data retrieval	Notification and business-event propagation
FoodRescue example	Checking donor verification	Publishing `OfferPublished`

The combination of these mechanisms allows FoodRescue TN to maintain direct communication when a decision is required while distributing business events without creating unnecessary dependencies between services.

6.2 Synchronous Interactions (REST / OpenFeign)

Synchronous communication is based on HTTP requests and REST APIs. Each microservice exposes endpoints that allow other services to retrieve information or execute authorised operations.

Within FoodRescue TN, OpenFeign is used as a declarative HTTP client. Instead of manually constructing HTTP requests, a service defines a Java interface containing the remote endpoints it needs to call. Spring Cloud OpenFeign generates the implementation and handles the communication.

Combined with Eureka, OpenFeign allows services to communicate using their registered service names rather than fixed IP addresses and ports. The API Gateway remains the single entry point for external client requests, while internal service-to-service calls can use service discovery directly.

6.2.1 Main synchronous interactions

The principal synchronous interactions between FoodRescue TN services are presented in Table 6.2.

Table 6.2: Main synchronous interactions between services

Requesting service	**Target service**	**Purpose**	**Expected result**
Donation Offer	Partner Registry	Check whether the donor is verified before accepting an offer.	Verification status.
Matching & Pickup	Donation Offer	Retrieve offer details and reserve the offer to prevent another active match.	Offer details and reservation confirmation.
Matching & Pickup	Partner Registry	Retrieve association location, capacity and accepted food categories.	Compatible association information.
Matching & Pickup	Volunteer Driver	Find an available driver near the pickup location.	Available driver information.
Marketplace	Donation Offer	Retrieve offer information when constructing a basket from donor stock, where required.	Offer details.

These interactions support important business decisions. For example, the Donation Offer service must verify the donor before publishing an offer, while the Matching service must obtain the information needed to select a suitable association and organise a pickup.

6.2.2 REST API contracts

The services expose REST endpoints with defined HTTP response codes. These codes allow the requesting service to distinguish successful operations from validation errors, missing resources and conflicts.

Table 6.3 presents the main endpoints involved in synchronous communication.

Note: The `POST /api/offers/{id}/lock` operation is a proposed internal operation representing the reservation required by the matching workflow. It is not part of the confirmed REST endpoint list yet. Its exact route and implementation must be agreed upon in the shared API contract before development.

Table 6.3: REST endpoints used in inter-service communication

Service	Endpoint	Method	Purpose
Offer	`/api/offers/{id}`	GET	Retrieve offer details.
Offer	`/api/offers/{id}/lock`	POST	Proposed internal operation to reserve an offer for matching.
Partner	`/api/partners/{id}`	GET	Retrieve partner information and verification status.
Matching	`/api/matches`	POST	Initiate a matching operation.
Driver	`/api/drivers/available?lat=&lng=`	GET	Find available drivers near a location.
Pickup	`/api/pickups/{id}/assign`	POST	Assign a driver to a pickup.
Pickup	`/api/pickups/{id}/status`	PATCH	Update the pickup status.

The assignment operation must also follow the agreed contract between the Matching and Volunteer Driver services. The Matching service remains responsible for the pickup assignment and delivery status.

6.2.3 Example of OpenFeign communication

For example, the Matching & Pickup service needs to retrieve the details of an offer from the Donation Offer service. Instead of accessing MongoDB directly, it uses an OpenFeign client.

A simplified Java interface is shown below:

```
@FeignClient(name = "donation-offer-service")
public interface OfferClient {

    @GetMapping("/api/offers/{id}")
    OfferDTO getOfferById(
        @PathVariable("id") String id
    );
}
```

The service can then call the remote endpoint through the interface:

```
OfferDTO offer = offerClient.getOfferById(offerId);
```

Eureka resolves the service name `donation-offer-service` to an available instance. The Matching service receives the response as an `OfferDTO`, without knowing how the offer is stored internally.

This approach preserves database independence and reduces the amount of communication code that developers must write.

6.3 Asynchronous Interactions (RabbitMQ Events)

Asynchronous communication is implemented through RabbitMQ, which acts as a message broker between the microservices. It allows one service to publish a business event while other services consume that event independently.

Unlike a synchronous request, the producer does not wait for every consumer to finish processing the event. The message broker receives the published message and routes it to the appropriate queues.

This mechanism is particularly useful in FoodRescue TN because several operations are triggered by a single business action. For example, when a food offer is published, the Matching service can start the matching process without requiring the Donation Offer service to execute the entire process itself.

6.3.1 RabbitMQ communication model

The asynchronous communication model relies on the following components:

- **Producer:** the microservice that publishes an event after a business action occurs.
- **Exchange:** receives published messages and routes them to queues according to the configured routing rules.
- **Queue:** temporarily stores messages until the corresponding consumer processes them.
- **Consumer:** the microservice that receives and processes messages from a queue.
- **Binding:** connects an exchange to a queue using routing rules.

For example, the Donation Offer service acts as the producer of `OfferPublished`, while the Matching & Pickup service acts as its consumer.

The following figure illustrates the general message flow.

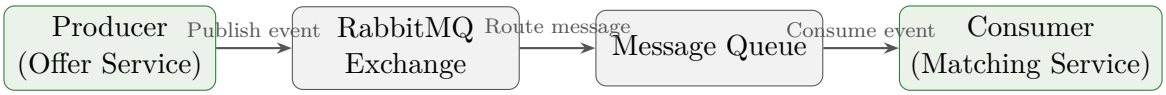


Figure 6.1: General asynchronous communication through RabbitMQ

6.3.2 Domain events in FoodRescue TN

A domain event represents a business fact that has already happened. It is not a direct command asking another service to execute an operation.

The events exchanged between the microservices are summarised in Table 6.4.

Table 6.4: Main RabbitMQ events in FoodRescue TN

Event	Producer	Consumer	Purpose
`OfferPublished`	Donation Offer	Matching & Pickup	Starts the matching process for a new food offer.
`MatchConfirmed`	Matching & Pickup	Notification	Notifies the association and the assigned driver.
`DeliveryCompleted`	Matching & Pickup	Impact Analytics	Records a completed donation delivery for impact calculation.
`RatingSubmitted`	Partner Registry & Trust	Impact Analytics	Updates the partner score and related impact information.
`BasketOrdered`	Surprise Basket Marketplace	Donation Offer	Reduces the available offer quantity after a confirmed basket order.
`BasketCollected`	Surprise Basket Marketplace	Impact Analytics	Records collected baskets for impact calculation.

Each event carries the information required by its consumers. For example, `DeliveryCompleted` must provide the identifiers and delivery information needed by Impact Analytics to calculate the corresponding impact indicators.

A common event structure can include an event identifier, event type, occurrence timestamp and business payload. The event identifier is especially important because it allows consumers to recognise messages that have already been processed.

6.3.3 Advantages of asynchronous communication

RabbitMQ provides several benefits to the architecture:

- **Loose coupling:** producers do not need to know the internal implementation of consumers.
- **Independent processing:** consumers process events according to their own availability and workload.
- **Improved fault isolation:** a temporary failure in Impact Analytics does not need to block the completion of a donation pickup.
- **Scalability:** consumers can be scaled independently when the number of incoming messages increases.
- **Event-driven workflows:** one business action can trigger several operations without creating a long chain of synchronous requests.

However, asynchronous communication introduces eventual consistency. For example, the global impact dashboard may not immediately reflect a delivery because the corresponding event must first be consumed and processed by Impact Analytics.

6.4 Sequence Diagrams

Sequence diagrams illustrate the chronological order of messages exchanged between actors and microservices. They make the communication mechanisms presented in the previous sections easier to understand by showing which service initiates an operation, which service responds and when events are published.

Two important workflows are presented: the donation process, from offer publication to impact calculation, and the surprise basket process, from ordering to collection.

6.4.1 Main flow: from offer to impact

The main workflow begins when a donor publishes surplus food. The Donation Offer service first checks the donor's verification status through the Partner Registry service. Once the offer is accepted and stored, it publishes an `OfferPublished` event through RabbitMQ.

The Matching & Pickup service consumes this event and starts the matching process. It retrieves and reserves the offer through the Offer service, obtains compatible association information from the Partner Registry, and searches for an available volunteer driver through the Driver service.

After a suitable association and driver have been identified, the Matching service creates the match and schedules the pickup. It then publishes `MatchConfirmed` through RabbitMQ. The Notification service consumes this event and sends notifications to the relevant participants.

When the driver completes the delivery, the Matching service publishes `DeliveryCompleted` through RabbitMQ. Impact Analytics consumes this event, records the delivered quantity and updates the indicators for kilograms saved, meals provided and avoided CO_2e.

The sequence is illustrated in Figure 6.2.

Communication interpretation:

- Solid arrows represent synchronous REST/OpenFeign calls.
- Dashed green arrows represent asynchronous RabbitMQ events.
- RabbitMQ acts as an intermediary between event producers and consumers.
- The Matching service coordinates the operational process, while Impact Analytics processes the completed delivery independently.

The separation between the operational workflow and impact calculation is important. The delivery can be completed even if the impact dashboard is temporarily unavailable. The event can be processed later when the consumer becomes available.

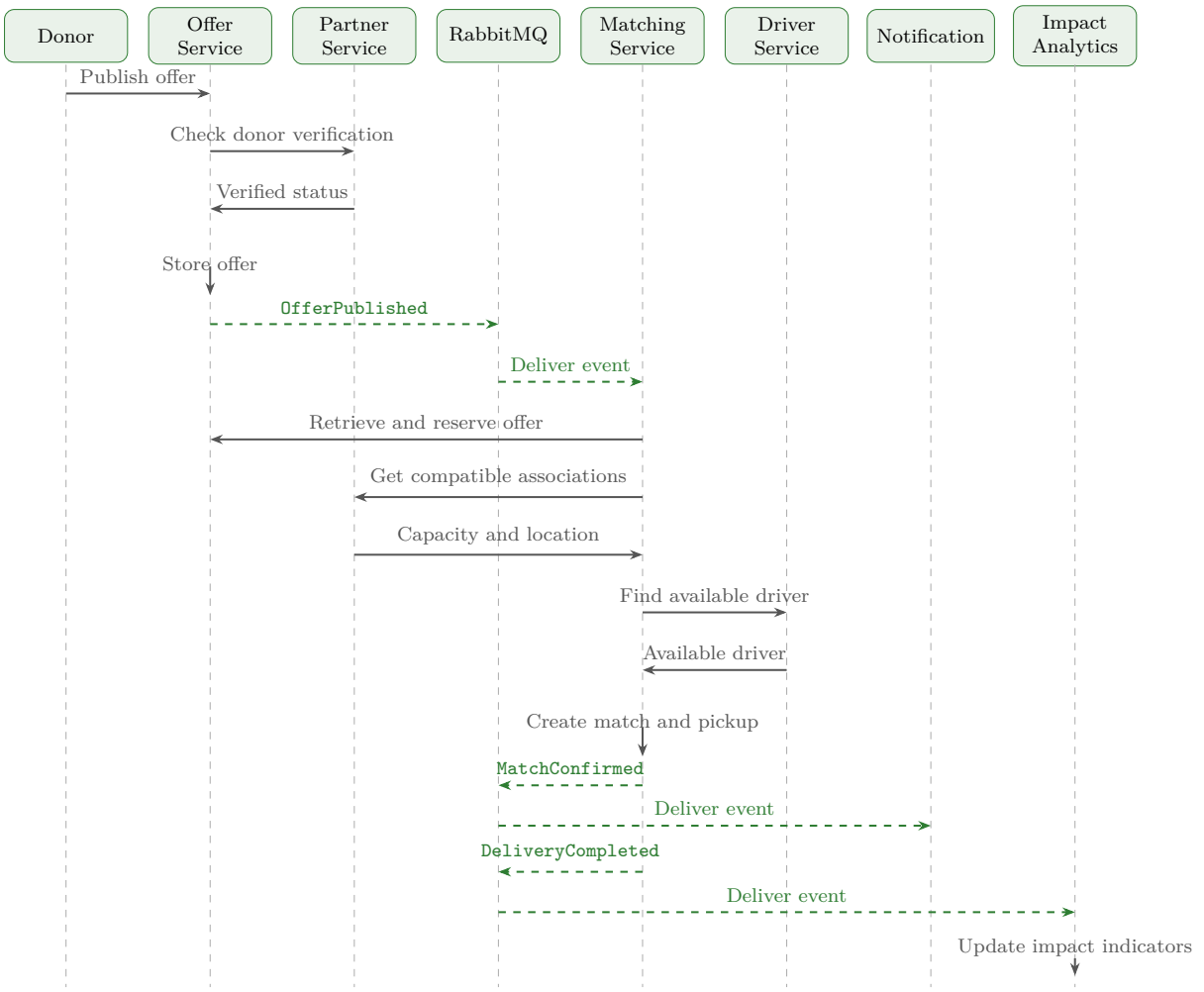


Figure 6.2: Main sequence: from offer publication to impact calculation

6.4.2 Surprise basket flow

The surprise basket workflow provides an alternative to food donation by allowing citizens to purchase discounted baskets prepared from available donor stock.

The process starts when a citizen browses the available baskets and selects one or more baskets. The Marketplace service checks the available quantity and processes the order in a database transaction. Payment is simulated in the current project scope, and a unique pickup code is generated for the confirmed order.

After confirmation, the Marketplace publishes a `BasketOrdered` event through RabbitMQ. The Donation Offer service consumes this event and decreases the corresponding `availableQuantity`, ensuring that the same stock is not simultaneously allocated to a donation and a basket sale.

When the citizen collects the basket using the pickup code, the Marketplace updates the order status to `COLLECTED` and publishes a `BasketCollected` event through RabbitMQ.

Finally, Impact Analytics consumes the event and adds the collected basket's weight to the impact indicators.

The complete interaction is shown in Figure 6.3.

Communication interpretation:

- The citizen communicates synchronously with the Marketplace to browse baskets, place an order and collect it.
- The Marketplace publishes `BasketOrdered` after successful order confirmation.
- RabbitMQ delivers the event to the Donation Offer service, which updates the remaining quantity of the original offer.
- The Marketplace publishes `BasketCollected` only after successful collection.
- RabbitMQ delivers the collection event to Impact Analytics, which updates the corresponding impact indicators.

The Marketplace and Donation Offer services maintain their own quantities. The Marketplace controls the available basket quantity during ordering, while the Offer service updates the remaining quantity of the original surplus through the event. This separation preserves database ownership while keeping the two representations consistent.

6.5 Reliability Aspects

In a distributed architecture, communication failures can occur even when each microservice works correctly. A service may become temporarily unavailable, a request may exceed its timeout, or a RabbitMQ message may be delivered more than once.

FoodRescue TN must therefore implement reliability mechanisms that protect the business data and allow the main workflows to continue despite temporary failures.

The reliability strategy focuses on the following aspects.

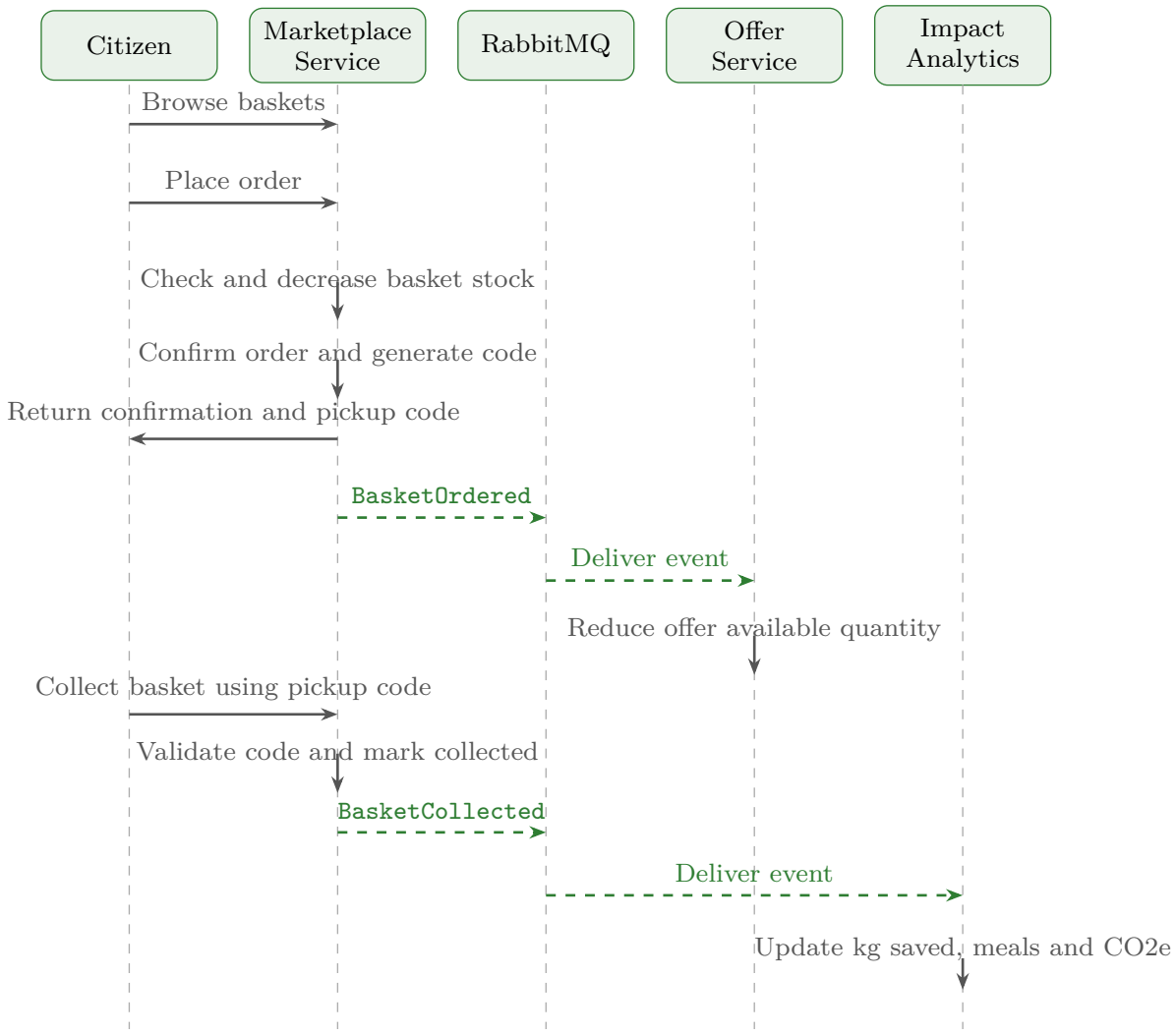


Figure 6.3: Surprise basket sequence: from order to impact calculation

6.5.1 Timeouts and error handling

A synchronous request depends on the availability of the target service. If the target service does not respond within an acceptable period, the requesting service should stop waiting and handle the failure.

For example, if the Matching service cannot contact the Partner Registry service, it must not assume that an association is verified or compatible. The matching operation should be temporarily rejected or deferred until the required information becomes available.

HTTP response codes also help distinguish different situations:

- **400 Bad Request:** the request contains invalid data.
- **403 Forbidden:** the user is not authorised to perform the operation.
- **404 Not Found:** the requested resource does not exist.
- **409 Conflict:** the operation conflicts with the current state of the resource, for example an offer already locked by another match.
- **5xx Server Errors:** the target service encountered an internal error or is temporarily unavailable.

Timeouts prevent services from waiting indefinitely. For temporary failures, controlled retries may be used, provided that repeating the operation cannot create duplicate business actions.

6.5.2 Message delivery and retries

RabbitMQ separates message production from message consumption. However, a consumer may be temporarily unavailable or fail while processing a message.

To handle these situations, consumers should acknowledge a message only after its processing has completed successfully. If processing fails, the message can be retried according to a controlled retry policy.

A message that repeatedly fails can be redirected to a **Dead Letter Queue (DLQ)** for later inspection and reprocessing. This prevents a problematic message from blocking the normal processing of other messages.

The proposed handling strategy is summarised below:

1. The producer publishes a business event to RabbitMQ.
2. RabbitMQ routes the message to the consumer queue.
3. The consumer receives and processes the message.
4. If processing succeeds, the consumer acknowledges it.

5. If a temporary failure occurs, the message is retried.

6. If the maximum number of retries is reached, the message is sent to the Dead Letter Queue for investigation.

6.5.3 Idempotent event consumption

RabbitMQ may deliver a message more than once, for example when a consumer completes its database operation but fails before acknowledging the message.

Without protection, processing the same event twice could increase the impact indicators twice or decrease the available offer quantity incorrectly.

FoodRescue TN therefore adopts **idempotent event consumption**. This means that processing the same event more than once must produce the same final result as processing it once.

As described in Chapter 5, the Impact Analytics service maintains a `processed_event` table containing the identifiers of events that have already been handled.

Before processing a new event, the service checks whether its identifier already exists. If it does, the event is ignored. Otherwise, the service inserts the event identifier, records the impact fact and updates the corresponding aggregates within the same database transaction.

The same principle must be applied to other consumers, such as the Donation Offer service consuming `BasketOrdered`, so that duplicate messages cannot decrease the available quantity more than once.

6.5.4 Data consistency and concurrent operations

Since every microservice owns its own database, a single business workflow cannot rely on one transaction covering all services.

For example, the Marketplace may successfully confirm a basket order before the Donation Offer service has processed the corresponding `BasketOrdered` event. During this short period, the two services may temporarily hold different representations of the available quantity.

This is an example of **eventual consistency**: the services become consistent after the event has been processed.

The following measures protect the business rules:

- **Database transactions:** each service uses local transactions to protect its own operations. For example, basket quantity reduction and order confirmation are handled atomically by the Marketplace.

- **Unique constraints:** database constraints prevent duplicate active matches, duplicate pickup assignments and repeated pickup codes.

- **Conditional updates:** the available basket quantity is checked and decreased atomically, preventing simultaneous orders from selling more baskets than are available.
- **Event identifiers:** consumers store processed event identifiers to avoid applying the same business event twice.
- **Controlled retries:** temporary communication failures are handled without immediately losing business events.

6.5.5 Reliability measures by service

Table 6.5 summarises the main reliability measures relevant to the FoodRescue TN microservices.

Table 6.5: Reliability measures applied to FoodRescue TN

Service	Potential failure	Reliability measure
Donation Offer	Duplicate event or concurrent offer reservation.	Idempotent event handling and controlled offer status transitions.
Matching & Pickup	Two matches for the same offer or duplicate driver assignment.	Database transactions, unique constraints and conflict responses (HTTP 409).
Partner Registry & Trust	Invalid partner information or repeated ratings.	Verification checks, validation rules and unique constraints.
Impact Analytics	Duplicate delivery or basket events.	The `processed_event` table and transactional aggregate updates.
Volunteer Driver	Overlapping availability slots or double booking.	Exclusion constraints and unique active assignment constraints.
Surprise Basket Marketplace	Overselling or duplicate collection.	Atomic quantity updates, unique pickup codes and order-status validation.
RabbitMQ consumers	Temporary failures or messages that cannot be processed.	Acknowledgements, controlled retries and a Dead Letter Queue.

These mechanisms do not eliminate every possible failure, but they reduce the risk of inconsistent data, duplicate operations and interrupted business workflows.

6.6 Chapter Conclusion

This chapter presented the communication mechanisms that connect the independent microservices of FoodRescue TN. The architecture combines synchronous REST/OpenFeign interactions, used when an immediate response is required, with asynchronous RabbitMQ events, used to propagate completed business actions without tightly coupling the services.

The main communication flows were illustrated through two sequence diagrams. The first follows a food donation from offer publication and partner verification to geographic matching, driver assignment, delivery and impact calculation. The second describes the surprise basket process, including order confirmation, stock synchronisation, collection and impact measurement.

Finally, the chapter examined the reliability challenges of distributed communication. Timeouts, error handling, controlled retries, message acknowledgements, idempotent consumers and database constraints help protect the system against temporary failures and duplicate operations. Since every microservice maintains its own database, eventual consistency is accepted and managed through event processing.

Together, these communication principles allow FoodRescue TN to coordinate its business services while preserving service independence, data ownership and the reliability of its main food rescue workflows.

Chapter 7

Overall Architecture

> *"A well-designed architecture provides a clear organization of the system, defines the responsibilities of its components, and ensures efficient communication between services."*

This chapter presents the overall architecture of FoodRescue TN. It provides a global view of the different components of the system and explains how they collaborate to deliver the main functionalities. It also describes the path followed by user requests and presents the deployment view of the application.

7.1 Global Architecture Diagram

The global architecture diagram provides an overview of the FoodRescue TN system. It illustrates the relationships between the client application, the API Gateway, the infrastructure components, the business microservices, and their databases.

FoodRescue TN follows a microservices architecture in which each business service is responsible for a specific domain. The services communicate through synchronous REST APIs or asynchronous RabbitMQ events, while each service manages its own data.

Figure 7.1 presents the overall architecture of the platform.

Diagram interpretation:

- Users interact with the platform through the client application.
- The API Gateway acts as the entry point for external requests and routes them to the appropriate backend service.
- Keycloak manages authentication and provides JWT tokens.
- Eureka allows services to register and discover other available service instances.
- The Config Server centralizes application configuration.
- RabbitMQ supports asynchronous communication between services through business events.
- Each business microservice owns and manages its own database.
- The shared User and Auth + Notification service handles application-level user information and notification operations. Keycloak remains responsible for identity and authentication.

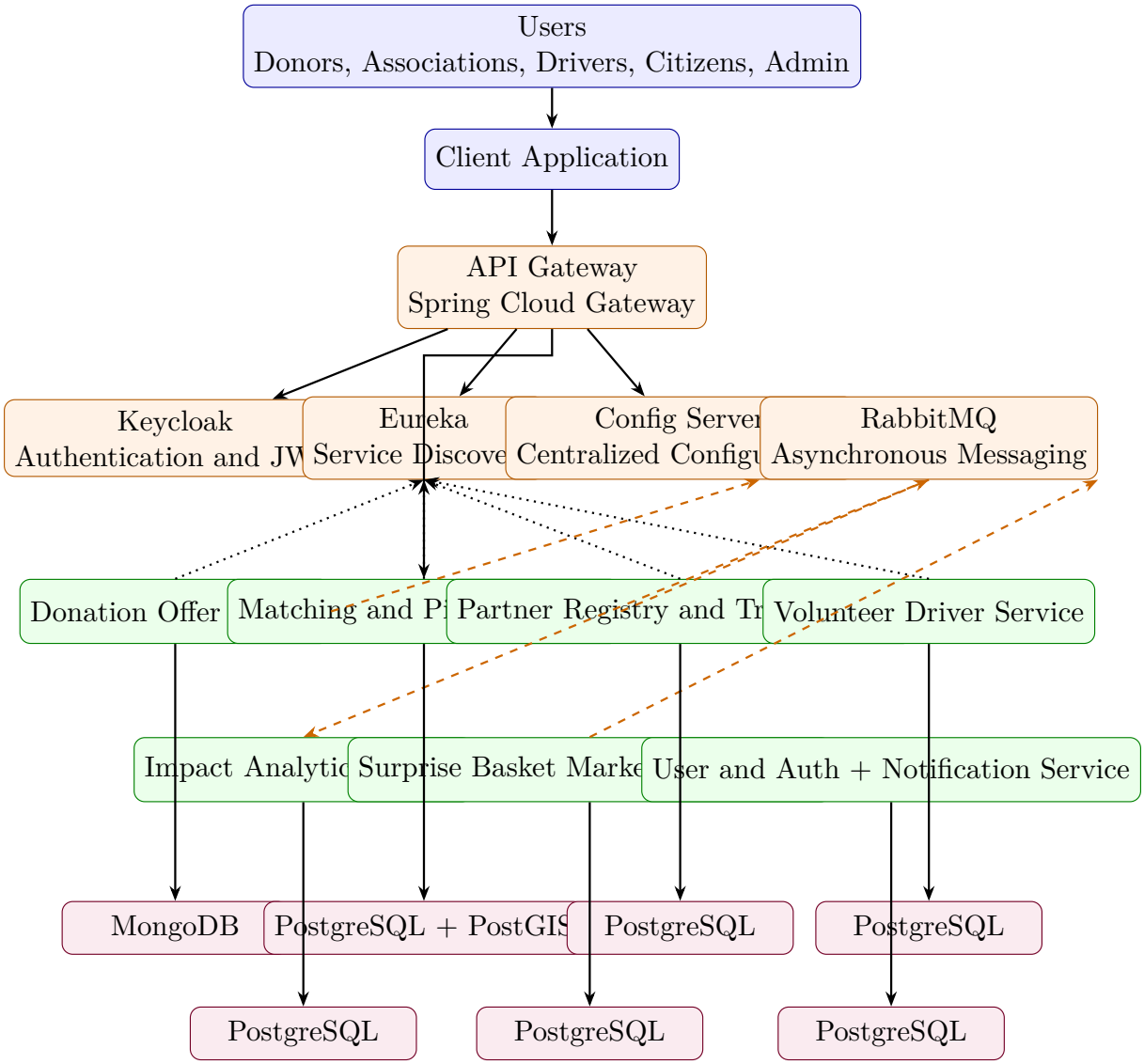


Figure 7.1: Overall architecture of FoodRescue TN

Note: The arrows in this diagram provide a logical overview of the architecture. They do not represent every individual REST call or RabbitMQ event. The detailed communication flows are presented in Chapter 6.

7.2 Explanation of the Request Path

FoodRescue TN follows a microservices architecture in which each business service is responsible for a specific part of the application. Users interact with the system through the client application, while the API Gateway manages incoming requests and forwards them to the appropriate microservice.

The request path describes how a request travels through the system, from the moment it is sent by a user until the expected response or action is produced.

1. User Request and API Gateway

The process starts when a user performs an action through the client application. For example, a donor may publish a new food donation offer, or a citizen may browse available surprise baskets.

The request is sent to the API Gateway, which acts as the single entry point to the backend system. Its main responsibilities are:

- Receiving incoming HTTP requests.
- Routing requests to the appropriate microservice.
- Applying security checks and handling authentication information.
- Returning the response to the client application.

This approach avoids exposing every microservice directly to external users.

2. Authentication and Authorization

Before accessing protected functionalities, users must be authenticated.

FoodRescue TN uses Keycloak to manage authentication and authorization. After a successful login, Keycloak provides a JSON Web Token (JWT) containing information about the authenticated user and their roles.

The client includes this token in its requests. The API Gateway and the concerned services validate the token and check whether the user has the required permissions.

For example, a donor can publish donation offers, while a volunteer driver can access and update the pickups assigned to them.

Authentication and authorization are therefore handled through a centralized identity provider, while the microservices enforce the permissions required for their own business operations.

3. Request Routing and Service Discovery

After receiving a valid request, the API Gateway identifies the microservice responsible for processing it.

Eureka is used as the service discovery component. Each microservice registers itself with Eureka, allowing other services to discover its available instance without depending on a fixed network address.

The Config Server centralizes the configuration of the different services. This makes it easier to manage shared configuration properties and maintain consistency across the application.

4. Business Processing

The request is then processed by the corresponding business microservice. Each service is responsible for its own business logic and manages its own data.

The six business microservices are:

- **Donation Offer Service:** manages food surplus offers published by donors.
- **Matching and Pickup Service:** handles the matching process and pickup organization.
- **Partner Registry and Trust Service:** manages associations, donors, verification, and ratings.
- **Volunteer Driver Service:** manages driver information and availability.
- **Impact Analytics Service:** calculates and presents environmental and social impact indicators.
- **Surprise Basket Marketplace Service:** manages discounted baskets and citizen orders.

In addition to these six business microservices, the architecture includes a shared User and Auth + Notification service. It supports application-level user information and notification operations. It does not represent an additional business microservice.

When a service needs information from another service immediately, it can communicate through synchronous REST calls using OpenFeign.

For operations that do not require an immediate response from another service, RabbitMQ is used to exchange events asynchronously. This allows services to remain independent while sharing important business information.

5. Data Management

Each microservice manages its own database according to its responsibilities. This principle ensures data ownership and reduces direct dependencies between services.

The main data storage technologies are:

- MongoDB for the Donation Offer Service.
- PostgreSQL with PostGIS for the Matching and Pickup Service.
- PostgreSQL for the Partner Registry and Trust Service.
- PostgreSQL for the Impact Analytics Service.
- PostgreSQL for the Volunteer Driver Service.
- PostgreSQL for the Surprise Basket Marketplace Service.
- PostgreSQL for application-level user data managed by the shared User and Auth + Notification service.

Keycloak manages identity information and authentication independently of the application databases.

A service accesses its own database directly. When another service needs information, it must communicate through the exposed API or through an event rather than accessing that service's database directly.

6. Response and Notification

After processing the request, the responsible service returns the result through the API Gateway to the client application.

For operations involving several services, some additional actions may continue asynchronously. For example, when a donation offer is published, an event can be sent to RabbitMQ to notify the Matching and Pickup Service that a new offer is available.

Similarly, when a pickup is completed, the Matching and Pickup Service publishes a `DeliveryCompleted` event. RabbitMQ delivers this event to the Impact Analytics Service to update the system's impact indicators.

The Notification functionality can also be triggered by relevant business events to inform users about important updates, such as a confirmed match or an assigned pickup.

Example: Publishing a Donation Offer

When a donor publishes a new food offer, the following steps take place:

1. The donor submits the offer through the client application.
2. The request passes through the API Gateway.

3. The system verifies the user's authentication and permissions using the JWT.
4. The Donation Offer Service checks the donor's verification status through the Partner Registry service.
5. The Donation Offer Service validates and stores the offer in MongoDB.
6. The service publishes an `OfferPublished` event through RabbitMQ.
7. The Matching and Pickup Service consumes the event and starts the matching process.
8. The Matching service retrieves the necessary association and driver information and organizes a pickup.
9. When the match is confirmed, the service publishes a `MatchConfirmed` event through RabbitMQ.

This example illustrates how synchronous requests and asynchronous events work together to support the main business processes of FoodRescue TN.

7.3 Deployment View

The deployment view describes how the different components of FoodRescue TN are organized and executed in the target runtime environment. Unlike the global architecture diagram, which focuses on the logical organization of the system, the deployment view focuses on the execution of services, databases, and infrastructure components.

FoodRescue TN is designed as a distributed application. Each microservice can run independently and communicate with other services through REST APIs or RabbitMQ messages.

1. Application Layer

The application layer contains the client application and the backend services.

The client application provides the interface through which donors, associations, volunteer drivers, citizens, and administrators interact with the system.

The backend is divided into independent Spring Boot microservices. Each service can be started, stopped, updated, and deployed separately without requiring the entire application to be redeployed.

The API Gateway receives external requests and forwards them to the appropriate backend service.

2. Infrastructure Layer

The infrastructure layer contains the components responsible for managing communication, service discovery, configuration, and security.

- **API Gateway:** provides a single entry point for external requests and routes them to the appropriate services.
- **Eureka Server:** maintains the registry of available microservice instances.
- **Config Server:** centralizes application configuration.
- **Keycloak:** manages user authentication and authorization.
- **RabbitMQ:** handles asynchronous communication and event delivery between services.
- **Docker:** packages application services and their dependencies into containers.
- **Docker Compose:** defines and starts the different application containers and their configurations.

These components provide the common infrastructure required by the business microservices.

3. Data Storage Layer

The data storage layer contains the databases used by the different microservices.

Each database belongs to a specific service and stores only the information required by that service. This separation allows each microservice to manage its data independently.

The following table summarizes the main deployment components and their responsibilities.

Table 7.1: Main components of the FoodRescue TN deployment

Component	**Technology**	**Responsibility**
API Gateway	Spring Cloud Gateway	Entry point and request routing
Service Discovery	Eureka	Registration and discovery of services
Configuration	Spring Cloud Config	Centralized configuration management
Authentication	Keycloak	Authentication and authorization

Continued on next page

Table 7.1 – continued from previous page

Component	**Technology**	**Responsibility**
Messaging	RabbitMQ	Asynchronous communication between services
Containerization	Docker	Packaging services and their dependencies into containers
Container Orchestration	Docker Compose	Defining and running the application containers
Donation Offer Service	Spring Boot + MongoDB	Management of food offers
Matching and Pickup Service	Spring Boot + PostgreSQL + PostGIS	Matching and pickup organization
Partner Registry and Trust Service	Spring Boot + PostgreSQL	Partner management, verification and ratings
Impact Analytics Service	Spring Boot + PostgreSQL	Impact calculation and data access
Volunteer Driver Service	Spring Boot + PostgreSQL	Driver management and availability
Surprise Basket Marketplace Service	Spring Boot + PostgreSQL	Basket management and citizen orders
User and Auth + Notification Service	Application service + PostgreSQL; Keycloak for identity	Application-level user information and notifications

4. Deployment Organization

The different components can be deployed as independent application instances. Each Spring Boot microservice runs separately and connects to its corresponding database.

The infrastructure services, including Eureka, Config Server, Keycloak, and RabbitMQ, are deployed as shared components that support communication and coordination across the application.

The databases are maintained separately to preserve the independence of the microservices. The Donation Offer Service uses MongoDB, while the other application services use PostgreSQL. The Matching and Pickup Service additionally uses PostGIS to support geographic operations.

Docker is used to package the different services and their dependencies into containers. Docker Compose provides a centralized configuration for defining and starting the application containers.

This approach simplifies the development and deployment environment because the required services can be started together using a common configuration.

Each microservice can also be scaled according to its individual workload. For example,

if the number of donation offers grows significantly, additional instances of the Donation Offer Service can be deployed without scaling every other service.

The exact production hosting environment can be selected according to the final implementation and infrastructure requirements.

5. Deployment Advantages

The proposed deployment organization offers several advantages:

- **Independent deployment:** each microservice can be deployed and updated separately.
- **Scalability:** services can be scaled according to their individual workload.
- **Maintainability:** separating business responsibilities makes the application easier to maintain.
- **Data independence:** each microservice controls its own database.
- **Communication flexibility:** REST and RabbitMQ support both synchronous and asynchronous interactions.
- **Centralized management:** shared infrastructure components simplify configuration, security, and service discovery.
- **Simplified deployment:** Docker and Docker Compose make it easier to package, configure, and start the different application components.

Conclusion

This chapter presented the overall architecture of FoodRescue TN, explaining the request path from the client application to the business microservices and their data storage components. It also described the deployment organization and the role of the main infrastructure components.

The proposed architecture provides a modular, scalable, and maintainable foundation for the platform. The six business microservices separate the main business responsibilities, while the shared User and Auth + Notification service supports common application functionalities.

By combining REST/OpenFeign, RabbitMQ, Keycloak, Eureka, Spring Cloud Config, Docker, and Docker Compose, FoodRescue TN can support its main operations, from publishing food surplus offers to organizing pickups, managing surprise basket orders, and measuring the resulting social and environmental impact.

The separation of databases and the use of asynchronous events also preserve service independence and support the reliability of the platform.

Chapter 8

Other Design Elements

This chapter presents the additional design elements required for the development and operation of FoodRescue TN. It focuses on the security mechanisms, business rules, development team organization, and technology stack adopted for the proposed system.

These elements complement the microservices architecture and service interactions presented in the previous chapters. They provide a common framework for implementing the platform consistently, securely, and reliably.

8.1 Security (Keycloak, JWT, Roles)

Security is an essential aspect of FoodRescue TN because the platform handles user accounts, partner information, donation offers, pickup operations, and marketplace orders.

The proposed system uses centralized authentication through Keycloak, JSON Web Tokens (JWT), and Role-Based Access Control (RBAC) to protect its resources and restrict access to authorized operations.

8.1.1 Authentication with Keycloak

Authentication is the process of verifying the identity of a user before allowing access to protected functionalities.

FoodRescue TN uses Keycloak as its centralized identity and access management solution. Keycloak is responsible for authenticating users and issuing access tokens after successful authentication.

The general authentication process is as follows:

1. The user attempts to access the application.
2. The application redirects the user to Keycloak for authentication.
3. The user provides their credentials.
4. Keycloak verifies the credentials.
5. After successful authentication, Keycloak issues a JWT access token.
6. The client includes this token in subsequent requests to protected resources.

This approach centralizes identity management and avoids implementing a separate authentication mechanism in every business microservice.

8.1.2 JSON Web Token (JWT)

A JSON Web Token (JWT) is a compact token used to securely transmit information about an authenticated user between application components.

After successful authentication, Keycloak generates a JWT that the client includes in the HTTP Authorization header when accessing protected APIs.

The general format is:

```
Authorization: Bearer <JWT_TOKEN>
```

The token contains claims such as:

- **Subject (sub):** identifies the authenticated user.
- **Issuer (iss):** identifies the token issuer, such as Keycloak.
- **Expiration (exp):** indicates when the token becomes invalid.
- **Roles:** provide information about the permissions assigned to the user.

The API Gateway validates incoming tokens before forwarding requests to protected resources. Protected microservices also enforce the necessary authorization checks.

JWT allows the different services to verify the user's identity without requiring the user to authenticate again for every request.

8.1.3 Role-Based Access Control (RBAC)

FoodRescue TN applies Role-Based Access Control (RBAC). This means that permissions are assigned according to the role of each user.

Each user receives an appropriate role that determines which functionalities they can access and which operations they are allowed to perform.

The main roles are:

- **DONOR:** represents bakeries, restaurants, hotels, and shops that publish surplus food offers and manage their donations.
- **ASSOCIATION:** represents charities and associations that receive suitable donations, organize their collection, and provide feedback.
- **DRIVER:** represents volunteer drivers who manage their availability and handle assigned pickups.
- **CITIZEN:** represents individuals who browse available surprise baskets and place orders.
- **ADMIN:** represents system administrators who manage users, verify partners, monitor platform activities, and supervise the overall operation.

Table 8.1 summarizes the main access permissions associated with each role.

8.1.4 Authorization and Access Control

Authorization determines whether an authenticated user has permission to perform a specific action.

For example:

- A donor can modify or cancel only their own donation offers.
- A driver can update the status of pickups assigned to them.
- A citizen can access and manage only their own basket orders.
- An administrator can access authorized administrative functionalities.
- Only authorized administrators can verify partners and perform restricted administrative operations.

These permissions are enforced through security configurations and authorization checks in the backend services.

The system also verifies resource ownership. Therefore, possessing a valid token alone does not allow a user to modify another user's information or resources.

8.1.5 Security Objectives

The proposed security mechanisms aim to achieve the following objectives:

- **Authentication:** ensure that users are properly identified.
- **Authorization:** restrict access according to user roles and permissions.
- **Confidentiality:** protect user and operational information from unauthorized access.
- **Integrity:** prevent unauthorized modification of data.
- **Centralized identity management:** manage authentication through Keycloak.

8.2 Business Rules and Assumptions

Business rules define the conditions and constraints that govern how FoodRescue TN operates. They ensure that the different actors interact with the platform in a consistent and organized way.

The following rules describe the main expected behaviors of the system.

8.2.1 User and Partner Management

- Every user must have an account to access protected functionalities.
- Each user is associated with an appropriate role.
- Donors and associations must provide the required information before using their partner functionalities.
- Partner verification is performed by an authorized administrator.
- Users can only perform operations permitted by their roles.
- Users can access or modify only the information and resources they are authorized to manage.

8.2.2 Donation Offer Management

- A donor can publish a food surplus offer by providing information such as the food category, quantity, description, pickup location, and pickup time window.
- An offer must contain all required information before it can be published.
- An offer must be available and eligible before it can participate in the matching process.
- A cancelled, expired, or already allocated offer must not be assigned again.
- During matching, an offer must be allocated in a controlled manner to prevent multiple associations from receiving the same offer.
- Donors are responsible for providing accurate information about their food offers.
- Offers must respect the applicable food safety and storage requirements.

8.2.3 Matching and Pickup Management

- The matching process identifies suitable associations according to criteria such as location, food category, capacity, and availability.
- Geographic proximity is considered when selecting a suitable association.
- A donation offer must not be assigned to multiple associations at the same time.
- A pickup can be assigned to an available volunteer driver.
- A driver must be informed about the pickup location and scheduled time.
- Only the driver assigned to a pickup, or another authorized user, can update its status.

- Pickup status must be updated as the operation progresses.
- A pickup is considered completed only after its completion has been confirmed.
- A completed pickup can contribute to the platform's impact indicators.

8.2.4 Surprise Basket Marketplace

- Donors can publish surprise baskets containing discounted surplus food.
- Each basket must include relevant information such as its description, price, quantity, and collection period.
- Citizens can place orders only for available baskets.
- The available quantity must be updated when an order is confirmed.
- The system must prevent orders from exceeding the available basket stock.
- A basket must not be ordered beyond its available quantity, even when multiple orders are submitted at the same time.
- Collection must be confirmed before an order is considered successfully collected.
- Repeated order events must not reduce the available stock more than once.

8.2.5 Ratings and Impact Calculation

- Associations can submit ratings after relevant donation operations have been completed.
- Ratings must be associated with the concerned partner and operation.
- Impact indicators must be based on recorded and completed operations.
- The quantity of food saved is calculated using the quantities confirmed through completed operations.
- The number of meals and the estimated CO_2 equivalent avoided are calculated using documented conversion factors.
- Cancelled or unsuccessful operations must not be counted as completed food-saving activities.
- The same completed operation must not be counted more than once in the impact calculations.
- Impact Analytics receives relevant completion events through RabbitMQ before updating its indicators.

8.2.6 Main Business Assumptions

The implementation of FoodRescue TN relies on several assumptions:

- Users have access to an internet connection to interact with the platform.
- Donors provide accurate information about the quantity, category, condition, and availability of surplus food.
- Associations maintain updated information about their location, capacity, and requirements.
- Volunteer drivers update their availability and pickup status.
- Geographic information is available to support proximity-based matching.
- The platform uses agreed conversion factors to estimate meals and environmental impact.
- The actual availability of food and the completion of a pickup must be confirmed by the concerned actors.

These rules and assumptions provide a common basis for implementing the business logic across the different microservices.

Specific operational thresholds, such as maximum pickup delays and food eligibility conditions, can be configured according to the platform's requirements.

8.3 Team Organization

The development of FoodRescue TN requires coordination between different development activities. Since the system is based on a microservices architecture, the work must be organized to ensure that the different services can be developed and integrated consistently.

The team organization is based on the main technical and functional responsibilities involved in the project.

8.3.1 Development Responsibilities

The main responsibilities can be organized as follows:

- **Project Coordination:** organizes development activities, defines priorities, follows project progress, and ensures that the different tasks remain consistent with the project objectives.
- **Backend Development:** implements the Spring Boot microservices, business logic, REST APIs, and communication mechanisms.

- **Frontend Development:** develops the client interface and integrates it with the backend APIs.
- **Database Management:** designs and manages the databases, data models, and relationships required by the different services.
- **Security and Integration:** configures Keycloak, JWT, API Gateway, service discovery, and communication between microservices.
- **Testing and Quality Assurance:** verifies the main functionalities, tests API endpoints, checks integration between services, and identifies errors.

These responsibilities describe the main work areas of the project. Depending on the team size, one member may be responsible for several activities, and multiple members may collaborate on the same task.

8.3.2 Work Organization

The project development is organized into several activities:

1. **Requirements Analysis:** identifying the actors, their needs, and the main functionalities of FoodRescue TN.
2. **System Design:** defining the microservices architecture, database structures, APIs, communication mechanisms, and security rules.
3. **Implementation:** developing the frontend, backend services, databases, and shared infrastructure components.
4. **Integration:** connecting the different microservices and verifying communication through REST APIs and RabbitMQ.
5. **Testing:** checking individual functionalities and the behavior of the complete system.
6. **Documentation:** preparing technical documentation describing the architecture, design decisions, and implementation.

8.3.3 Collaboration and Coordination

Because the system contains several independent microservices, coordination is important to avoid integration problems.

The team must agree on common API contracts, data formats, event structures, naming conventions, and security rules before integrating the services.

Regular communication and shared documentation help team members understand the responsibilities of each component and identify dependencies between development tasks.

This organization encourages parallel development while maintaining consistency across the complete FoodRescue TN system.

8.4 Technology Stack Summary

FoodRescue TN relies on a combination of technologies selected according to the responsibilities of each component. The technology stack supports backend development, data storage, communication, authentication, deployment, and analytics.

The following table summarizes the main technologies used in the proposed system.

8.4.1 Database Allocation

Each business microservice manages its own data according to its functional responsibilities.

The selected database allocation is summarized below:

- **Donation Offer Service:** MongoDB is used to store donation offer information.
- **Matching and Pickup Service:** PostgreSQL with PostGIS is used to manage matching operations, pickups, and geographic data.
- **Partner Registry and Trust Service:** PostgreSQL is used to store partner information, verification details, and ratings.
- **Impact Analytics Service:** PostgreSQL is used to store operational data and calculated impact indicators.
- **Volunteer Driver Service:** PostgreSQL is used to manage driver information, availability, and assignments.
- **Surprise Basket Marketplace Service:** PostgreSQL is used to manage baskets, orders, and collection information.
- **Shared User and Auth + Notification Service:** application-level user information and notification data are managed separately from the identity management responsibilities handled by Keycloak.

Each microservice is responsible for accessing and managing its own database. Other services interact with it through the defined APIs or asynchronous events rather than accessing its database directly.

8.4.2 Technology Selection

The selected technologies provide complementary capabilities:

- **Spring Boot** provides the foundation for developing independent backend microservices.
- **Spring Cloud** provides infrastructure components such as the API Gateway, Eureka, and Config Server.
- **Keycloak and JWT** provide centralized identity management and secure access to application resources.
- **RabbitMQ** supports asynchronous communication and event-driven interactions between independent services.
- **MongoDB and PostgreSQL** provide different data storage capabilities according to the needs of each service.
- **PostGIS** supports geographic operations required by the matching process.
- **Docker and Docker Compose** facilitate application containerization and the deployment of the different components.
- **Power BI** supports the visualization and analysis of the platform's impact indicators.
- **Python and FastAPI** are considered optional technologies for possible forecasting extensions and are not required for the core microservices architecture.

The shared User and Auth + Notification application service supports application-level user information and notification functionalities. Keycloak remains responsible for centralized identity management and authentication.

The proposed architecture contains six business microservices, together with shared infrastructure and application components. These shared components support the business services without being considered additional business microservices.

Table 8.1: Main user roles and permissions

Role	**Main permissions**
DONOR	Publish, update, and cancel food offers; manage surplus information.
ASSOCIATION	View suitable offers, participate in donation matching, manage received donations, and submit ratings.
DRIVER	Manage availability, view assigned pickups, and update pickup status.
CITIZEN	Browse available surprise baskets, place orders, and confirm collection.
ADMIN	Manage users, verify partners, supervise activities, and monitor system information.

Table 8.2: Technology stack of FoodRescue TN

Category	**Technology**	**Purpose**
Backend Development	Java and Spring Boot	Development of the business microservices and REST APIs.
API Gateway	Spring Cloud Gateway	Routing and management of incoming requests.
Service Discovery	Eureka	Registration and discovery of microservices.
Configuration Management	Spring Cloud Config	Centralized configuration of services.
Authentication	Keycloak and JWT	User authentication and role-based access control.
Synchronous Communication	REST and OpenFeign	Direct communication between microservices.
Asynchronous Communication	RabbitMQ	Event exchange between independent services.
Document Database	MongoDB	Storage of donation offer information in the Donation Offer Service.
Relational Database	PostgreSQL	Storage of structured business data for the relational services.
Geographic Data	PostGIS	Geographic queries and location-based matching in the Matching and Pickup Service.
Deployment	Docker and Docker Compose	Containerization and deployment of the application components.
Analytics and Reporting	Power BI	Visualization of operational, social, and environmental indicators.
Optional Forecasting	Python and FastAPI	Possible integration of forecasting functionalities in future extensions.

General Conclusion

The increasing amount of food waste represents a major economic, social, and environmental challenge. At the same time, many individuals and organizations continue to face food insecurity. Addressing this issue requires more than simply redistributing surplus food; it requires an organized, reliable, and accessible system capable of connecting food donors with the organizations and individuals who can benefit from their resources.

In this context, **FoodRescue TN** was proposed as a digital solution aimed at reducing food waste and encouraging solidarity through technology. The main objective of the project is to facilitate the collection, redistribution, and valorization of surplus food by connecting donors, associations, volunteer drivers, and citizens through a centralized digital ecosystem.

Throughout this report, we presented the different aspects of the proposed system, beginning with the analysis of the existing problem, the identification of stakeholders, and the definition of functional and non-functional requirements. We then translated these requirements into a structured system design based on a microservices architecture. This architecture is organized around six business microservices: Donation Offer, Matching and Pickup, Partner Registry and Trust, Impact Analytics, Volunteer Driver, and Surprise Basket Marketplace. Each service has a specific responsibility and manages its own data, promoting modularity, maintainability, and independent development.

The proposed architecture also integrates shared infrastructure components and communication mechanisms to ensure effective coordination between services. Spring Cloud Gateway, Eureka, and Spring Cloud Config support request routing, service discovery, and centralized configuration, while REST and OpenFeign enable synchronous communication. RabbitMQ supports asynchronous event-driven interactions, allowing important business events to be communicated between services without creating unnecessary dependencies. Furthermore, Keycloak and JSON Web Tokens provide centralized authentication and role-based access control, helping protect the platform and its resources.

Beyond its technical architecture, FoodRescue TN aims to create practical value through two complementary approaches. The donation management process facilitates the redistribution of surplus food to associations according to criteria such as location, category, capacity, and availability. In parallel, the Surprise Basket Marketplace provides citizens with the opportunity to purchase discounted surplus food, encouraging responsible consumption while offering donors an additional way to reduce food losses. The Impact Analytics Service complements these activities by providing indicators that help monitor the platform's social and environmental contribution, including quantities of food saved, estimated meals, and CO_2 equivalent avoided.

The report also established the business rules, security requirements, development responsibilities, and technology choices necessary to maintain consistency across the proposed system. Together, these design elements provide a coherent foundation for the

future implementation and integration of the platform.

Nevertheless, the proposed solution also offers opportunities for further development. Future improvements could include advanced matching algorithms, demand forecasting, intelligent recommendations, mobile application enhancements, real-time notifications, and integrations with additional food donors and charitable organizations. The platform could also evolve through the introduction of more advanced analytics and monitoring capabilities, enabling stakeholders to make better-informed decisions and evaluate the impact of their activities more effectively.

Ultimately, **FoodRescue TN is more than a food redistribution application; it represents a step toward a more connected, responsible, and sustainable food ecosystem in Tunisia.** By combining digital innovation, coordinated logistics, and social engagement, the project seeks to transform surplus food into an opportunity for solidarity and environmental responsibility. Its long-term ambition is to contribute to a future in which less food is wasted, more resources are shared, and technology becomes an effective tool for addressing real societal challenges.

Conclusion

This chapter presented the additional design elements of FoodRescue TN. It described the security mechanisms based on Keycloak, JWT, and Role-Based Access Control, as well as the business rules and assumptions governing the main platform operations.

It also introduced the organization of the development team and summarized the technologies selected for the implementation of the system, including backend development, databases, communication, deployment, and analytics.

Together with the architecture and interaction design presented in the previous chapters, these elements provide a consistent foundation for the implementation and integration of the FoodRescue TN system.

Appendix A

Event Contract Examples